ITCS213 Computer Organization and Architecture

Class: 1-2025
Sec 2 Wed 9:00-12:00 IT311
Sec 3 Thu 9:00-12:00 IT311
Sec 1 Thu13:00-16:00 IT311

Unit 5: **MIPS Assembly**, **Cache**, **Memory**, and **Integer Representations**

Lecturer: Dr. Peter M.D. Scully
Faculty of ICT, Mahidol University
peter.scu@mahidol.edu

Slides adapted from Subject Coordinator: Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Announcement: ITCS213

Unit 5: MIPS Assembly, Memory Hierarchy, Integer Representation

Announcements

- **Homework: (1)** Review Lab 1 Solution Examples. **(2)** Quiz Unit 5.

Today we aim to:

1. *Learn how MIPS handles data:*
   a. *How Cache and Memory Hierarchies speed-up execution & its problems*
   b. *How data moves from RAM, through Cache, into Registers.*

2. *Understand Registers (Load / Store operations)*
   a. *Words, Word Addresses*
   b. *Word Alignment as multiples of 4 (32-bits, 4-bytes) and its exceptions.*

3. *Understand:*
   a. *Integer Signed, Unsigned operation differences (add vs. addu)*
      - *Why MIPS choice for Integer add is addu.*
      - *Flags & Trap Exceptions (Carry, Borrow, Overflow)*
      - *and why "Bits Have No (inherent) Meaning":*

Agenda

Part : Reminder MIPS Assembly - Simple

Part : Memory Hierarchy

Part : MIPS with Cache & Memory

Part : Assembly Programming

Part : Integer Unsigned & Integer Signed

Part : Carry/Overflow Examples

Computer Organization Study Process:

How we build understanding in this unit

Step 1: **Reminder MIPS Assembly - Simple**

Instruction formats, registers, syscalls

Step 2: **Memory Hierarchy**

Cache, RAM, memory organization

Step 3: **MIPS with Cache & Memory**

Load/store instructions, addressing

Step 4: **Assembly Programming**

MIPS programming patterns, integer ops

Step 5: **Integer Representation**

Unsigned and signed integers, range

Step 6: **Carry & Overflow**

Arithmetic edge cases, worked examples

All steps are covered in this unit (Unit 5).

Grading

ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

GRADING SYSTEM				
Grade Criteria		A	>=	80
		B+	>=	75
		B	>=	70
		C+	>=	65
		C	>=	60
		D+	>=	55
		D	>=	50
		F	<	50

Content

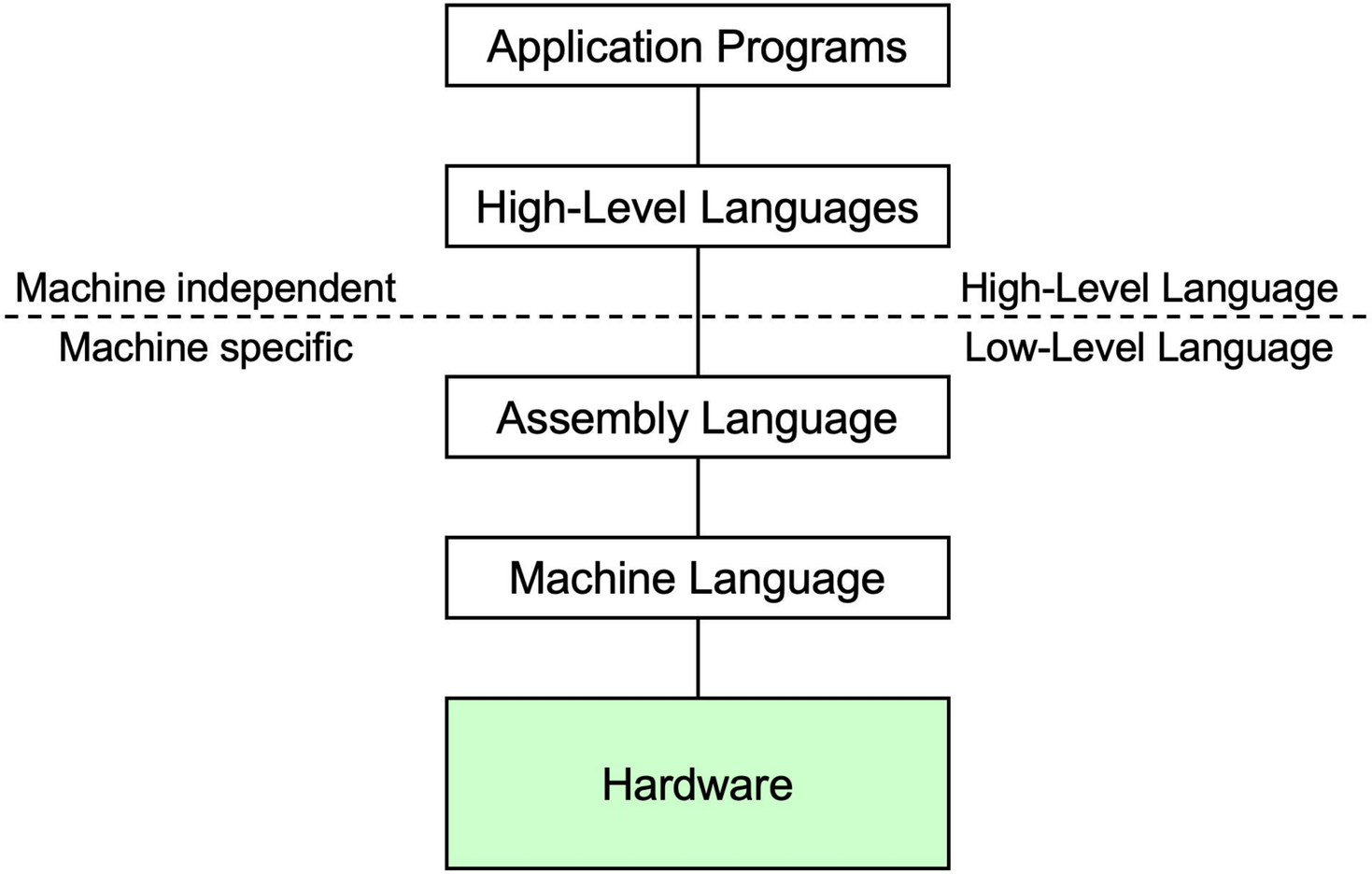
- **Assembly** Language
- **Memory**
- **MIPS** with **Cache** and **Memory**
- R-Type Instruction Format
- R-type Arithmetic, Logical, and Shift Instructions
- I-Type Instruction Format and Immediate Constants
- I-Type Arithmetic and Logical Instructions
- Pseudo Instructions

PART A

Reminder: MIPS Assembly Simple

Reminder of first steps in MIPS assembly: writing and running simple programs.

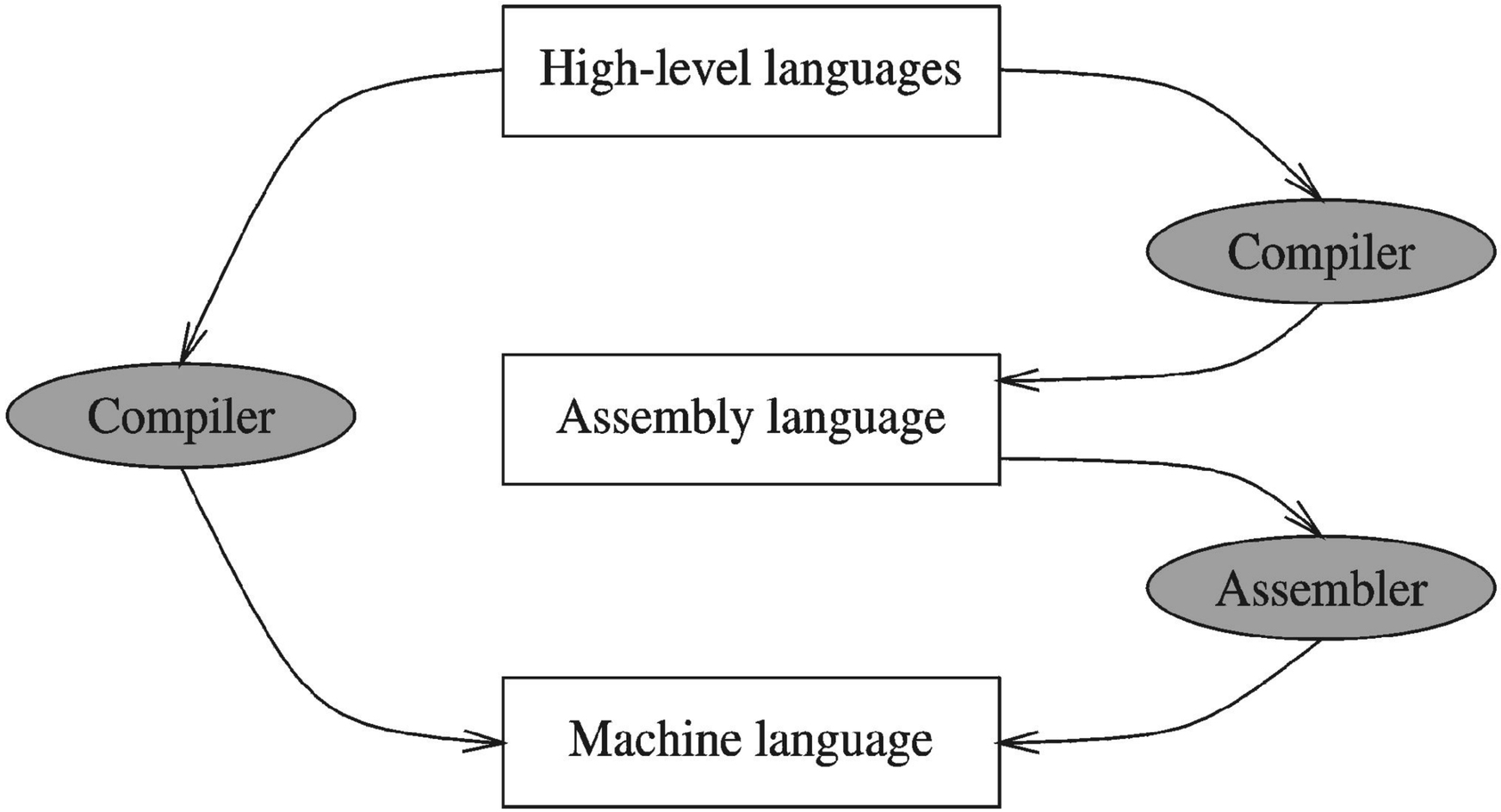
Hierarchy of Languages



Assembly and Machine Language

- Machine language
  - Native to a processor: executed directly by hardware.
  - Instructions consist of binary code: 1s and 0s.
- **Assembly** language
  - Slightly higher-level language.
  - The readability of instructions is better than machine language.
  - One-to-one correspondence with machine language instructions.
- Assemblers translate **assembly** to machine code.
- Compilers translate high-level programs to machine code
  - Either directly,
  - or indirectly via an assembler.

Compiler and Assembler



Instructions and Machine Language

- Each command of a program is called an instruction (it instructs the computer what to do).
- Computers only deal with binary data, hence the instructions must be in binary format (0s and 1s) .
- The set of all instructions, in binary format, makes up the computer's machine language.
    - This is also referred to as the instruction set.

Instruction Fields

- Machine language instructions are usually made up of several fields.
  - Each field specifies different information for the computer.

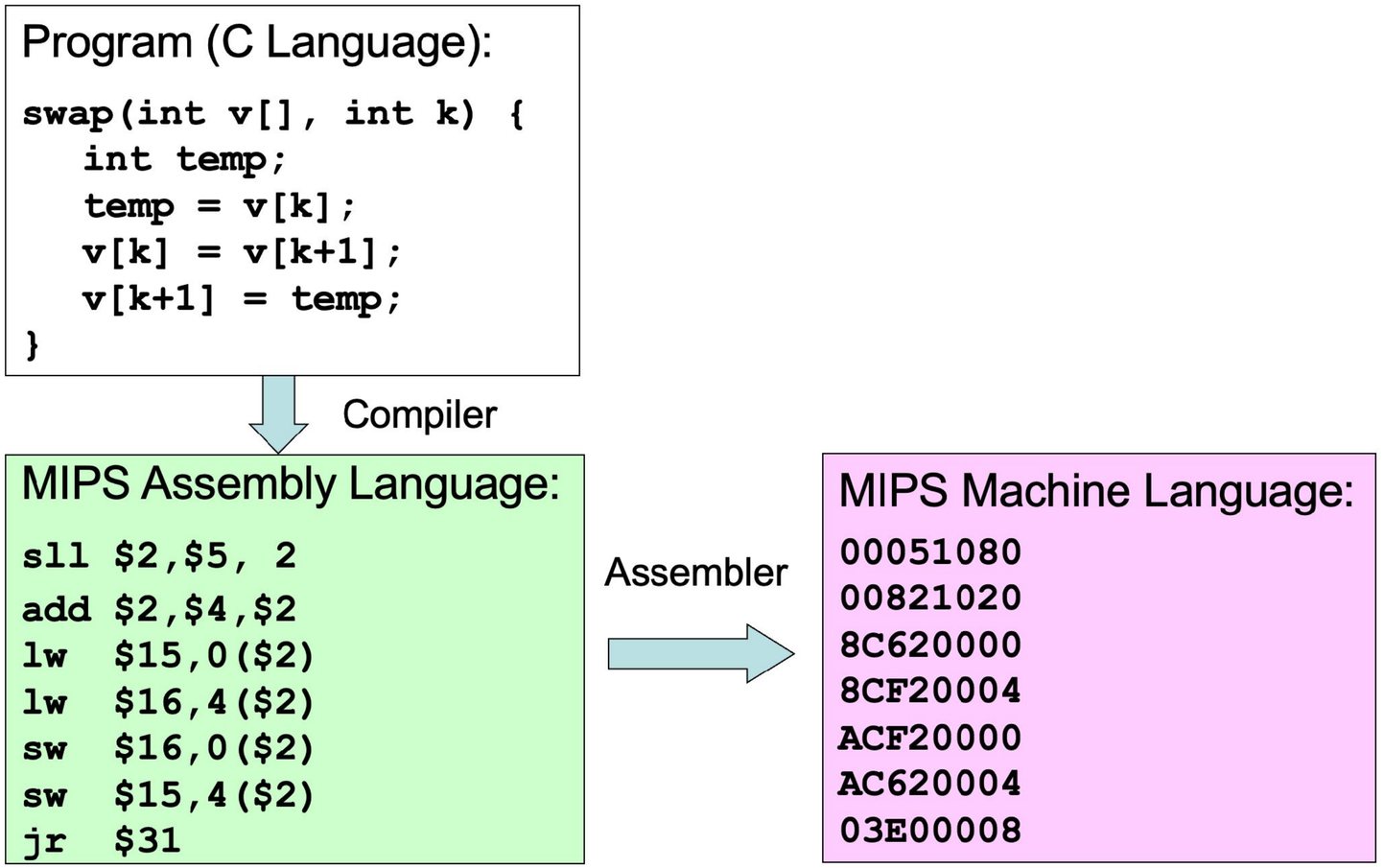
Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

Instruction Fields

- The two major fields are:
  - Opcode field, which stands for operation code, specifies the particular operation to be performed.
    - Each operation has its unique opcode.
  - Operand fields specify where to get the source and destination operands for the operation specified by the opcode.
    - The source/destination of operands can be
      - a constant,
      - the **memory**, or
      - one of the general-purpose registers.

Translating Languages

- A statement in a high-level language is translated typically into several machine-level instructions.
  - 1:M relationship



Advantages of High-Level Languages

- Program development is faster,
  - high-level statements: fewer instructions to code.
- Program maintenance is easier,
  - for the same above reason.
- Programs are portable,
  - contain few machine-dependent details,
  - can be used with little or no modifications on different machines.
    - Compiler translates to the target machine language.
  - Note: **Assembly** language programs are not portable.

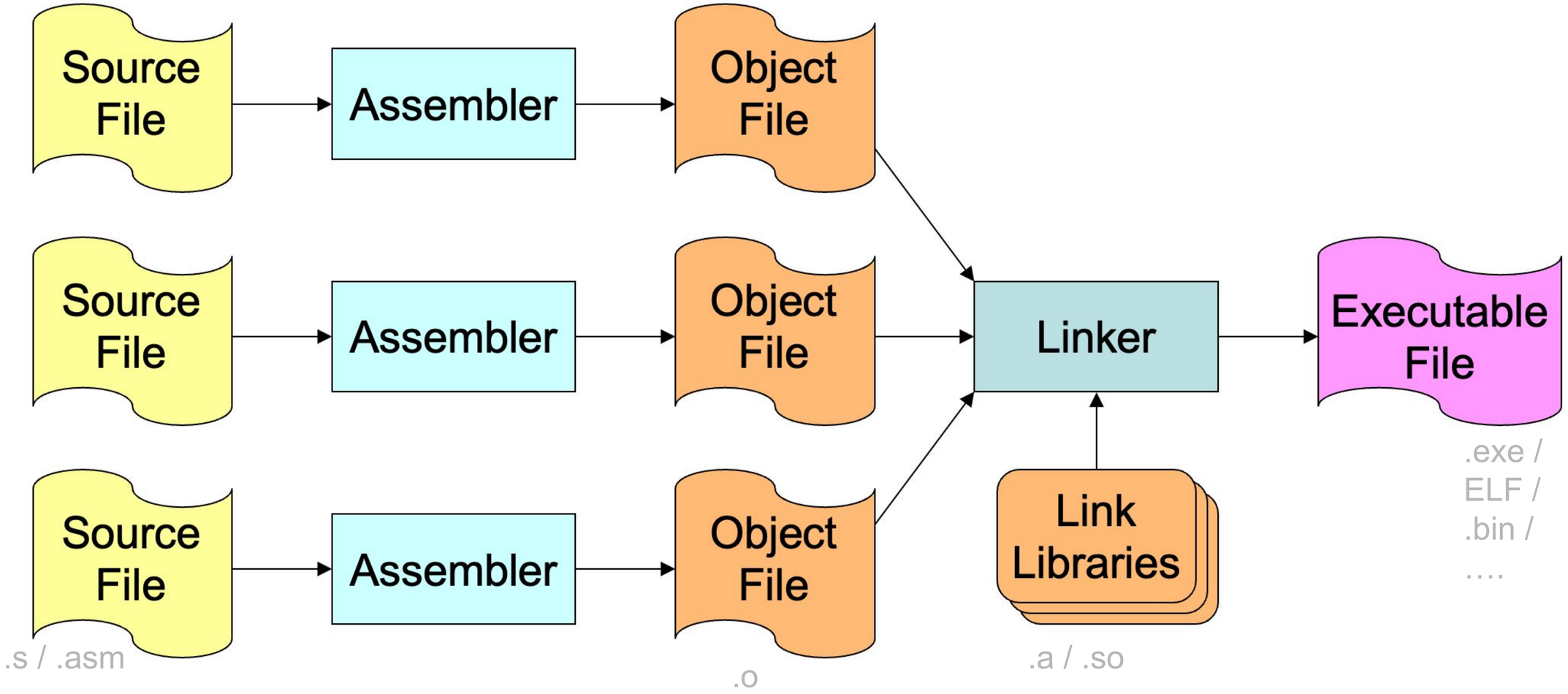
Why Assembly Language?

- Many reasons:
  - accessibility to system hardware,
  - space and time efficiency.
- Accessibility to system hardware:
  - **Assembly** language is useful for implementing system software,
  - also useful for small embedded system applications.
- Space and Time efficiency:
  - understanding sources of program inefficiency,
  - tuning program performance,
  - writing compact code.

Assembly Language Programming Tools

- Editor:
  - allows us to create and edit **assembly** language source files.
- Assembler:
  - converts **assembly** language programs into object files,
  - object files contain the machine instructions.
- Linker:
  - combines object files created by the assembler with link libraries,
  - produces a single executable program.
- Debugger:
  - allows us to trace the execution of a program,
  - allows us to view machine instructions, **memory**, and registers.

Assemble and Link Process



- A project may consist of multiple source files.
- Assembler translates each source file separately into an object file.
- Linker links all object files together with link libraries into an executable file.

(Note: MARS/ SPIM Simulators do this all in RAM)

MARS Assembler and Simulator Tool

/Users/phoenix/Desktop/MIPS Assembly Programming/MIPS/Chatnarin.asm - MARS 4.5

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Chatnarin.asm

```
1  # Title: [Double an Integer]
2  # Author: [Chatnarin Chantra]
3  # Date: [28/8/2025]
4  # Description: [print double that number using the add instruction]
5  # Input: 20
6  # Output:Enter an integer: 20
7  # Double of your number is: 40
8  .data
9  # Your variables here
10 prompt: .asciiz "Enter an integer: "
11 result: .asciiz "Double of your number is: "
12 newline: .asciiz "\n"
13 .text
14 .globl main
15 main:
16 # Your code here
17 # Print
18 li $v0, 4
19 la $a0, prompt
20 syscall
21 # Read int
22 li $v0, 5
23 syscall
24 move $t0, $v0
25 # Double the number
26 add $t1, $t0, $t0 # Print result
27 li $v0, 4
28 la $a0, result
29 syscall
30 # Print doubled num
31 li $v0, 1
32 move $a0, $t1
33 syscall
34 # Print newline
35 li $v0, 4
36 la $a0, newline
37 syscall
38 # Exit program
39 li $v0, 10
40 syscall
```

Line: 28 Column: 15 ✓ Show Line Numbers

Mars Messages Run I/O

```
Enter an integer: 3
Double of your number is: 6

-- program is finished running --
```

Clear

Registers Coproc 1 Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x1001002e
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000003
$t1	9	0x00000006
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400054
hi		0x00000000
lo		0x00000000



Mars4_5.jar

Java JAR file - 4.2 MB

Information Show More

Created	Yesterday, 12:56
Modified	Yesterday, 12:56
Last opened	Today, 12:48

Tags

Add Tags...

MARS Assembler and Simulator Tool

- Simulates the execution of a **MIPS** program,
  - no direct execution on the underlying Intel processor.
- Editor with color-coded **assembly** syntax.
- Assembler:
  - converts **MIPS assembly** language programs into object files.
- Debugger:
  - allows us to trace the execution of a program and set breakpoints,
  - allows us to view machine instructions, edit registers and **memory**.

MIPS: Load Immediate

- Initialization (Register Loading)

- li (Load Immediate): Loads a constant value into a register.
- $t1 (temporary register 1) is set to 5.
- $t2 (temporary register 2) is set to 7.
- Note:
  - In the comments, R2 and R3 refer to the logical registers in your original question (mapped to $t1 and $t2 in MIPS).

▼ Code 402 char

```
    .text
    .globl main

main:
    # Load values into registers (R2=$t1, R3=$t2)
    li $t1, 5       # R2 = 5
    li $t2, 7       # R3 = 7

    # Perform ADD R1 = R2 + R3
    add $t0, $t1, $t2   # R1 = $t0 = 5 + 7

    # Print result (R1)
    li $v0, 1       # syscall: print integer
    move $a0, $t0   # argument = R1
    syscall

    # Exit program
    li $v0, 10      # syscall: exit
    syscall
```

MIPS: Add Integer R-Type

- Addition Operation

  add $t0, $t1, $t2 # R1 = $t0 = 5 + 7

- add: Adds the values of $t1 (5) and $t2 (7), storing the result in $t0 (temporary register 0).
- Result: $t0 = 12 (which is referred to as R1 in the comments).

Code 402 char

```
    .text
    .globl main

main:
    # Load values into registers (R2=$t1, R3=$t2)
    li $t1, 5       # R2 = 5
    li $t2, 7       # R3 = 7

    # Perform ADD R1 = R2 + R3
    add $t0, $t1, $t2   # R1 = $t0 = 5 + 7

    # Print result (R1)
    li $v0, 1       # syscall: print integer
    move $a0, $t0   # argument = R1
    syscall

    # Exit program
    li $v0, 10      # syscall: exit
    syscall
```


Print result (R1)
li $v0, 1 # syscall: print integer
move $a0, $t0 # argument = R1
syscall

Exit program
li $v0, 10 # syscall: exit
syscall

MIPS: Print Integer (1)

- Print the Result

```
li $v0, 1          # Prepare to print an integer
move $a0, $t0      # Set argument = $t0 (R1 = 12)
syscall            # Execute the print
```

- li $v0, 1: Loads the syscall code 1 into $v0 (MIPS uses $v0 to determine the syscall type).
  - 1 = “print integer” service.
- move $a0, $t0: Copies the value of $t0 (12) to $a0 (the register for syscall arguments).
- syscall: Triggers the syscall, printing the integer (12) to the console.

▼ Code 402 char

```
    .text
    .globl main

main:
    # Load values into registers (R2=$t1, R3=$t2)
    li $t1, 5        # R2 = 5
    li $t2, 7        # R3 = 7

    # Perform ADD R1 = R2 + R3
    add $t0, $t1, $t2   # R1 = $t0 = 5 + 7

    # Print result (R1)
    li $v0, 1        # syscall: print integer
    move $a0, $t0    # argument = R1
    syscall

    # Exit program
    li $v0, 10       # syscall: exit
    syscall
```


Print result (R1)
li $v0, 1 # syscall: print integer
move $a0, $t0 # argument = R1
syscall

Exit program
li $v0, 10 # syscall: exit
syscall

MIPS: Exit Program (10)

- Exit the Program

```
li $v0, 10      # Prepare to exit
syscall         # Terminate the program
```

- li $v0, 10: Loads the syscall code 10 into $v0.
  - 10 = “exit” service.
- syscall: Ends the program execution.

Code 402 char

```
    .text
    .globl main

main:
    # Load values into registers (R2=$t1, R3=$t2)
    li $t1, 5       # R2 = 5
    li $t2, 7       # R3 = 7

    # Perform ADD R1 = R2 + R3
    add $t0, $t1, $t2   # R1 = $t0 = 5 + 7

    # Print result (R1)
    li $v0, 1       # syscall: print integer
    move $a0, $t0   # argument = R1
    syscall

    # Exit program
    li $v0, 10      # syscall: exit
    syscall
```

MIPS: Registers & Syscall

MIPS Register Conventions:

- **$t0–$t9**: Temporary registers (caller-saved).
- **$v0**: Holds syscall numbers and return values.
- **$a0–$a3**: Store arguments for syscalls/ functions.

```
# Print result (R1)
li $v0, 1       # syscall: print integer
move $a0, $t0   # argument = R1
syscall
```

```
# Exit program
li $v0, 10      # syscall: exit
syscall
```

Syscalls:

- MIPS uses **syscall** to interact with the **OS**, e.g.,
  - Program termination = **10**
  - Print Integer = **1**
  - I/O, = ….



Key Points: Assembly & Machine Language

- **Machine language: binary instructions executed directly by hardware**
  - Opcode field specifies the operation;
  - operand fields identify data
- **Assembly provides human-readable mnemonics for instructions**
  - Assembler translates to machine code;
  - compiler handles high-level languages
- **MIPS development uses MARS simulator for testing and debugging**
  - Supports assembling, executing, and stepping through MIPS programs
- **Key MIPS instructions:**
  - **li (load immediate),**
  - **add,**
  - **sub,**
  - **syscall -** provides system services: print, read input, exit program

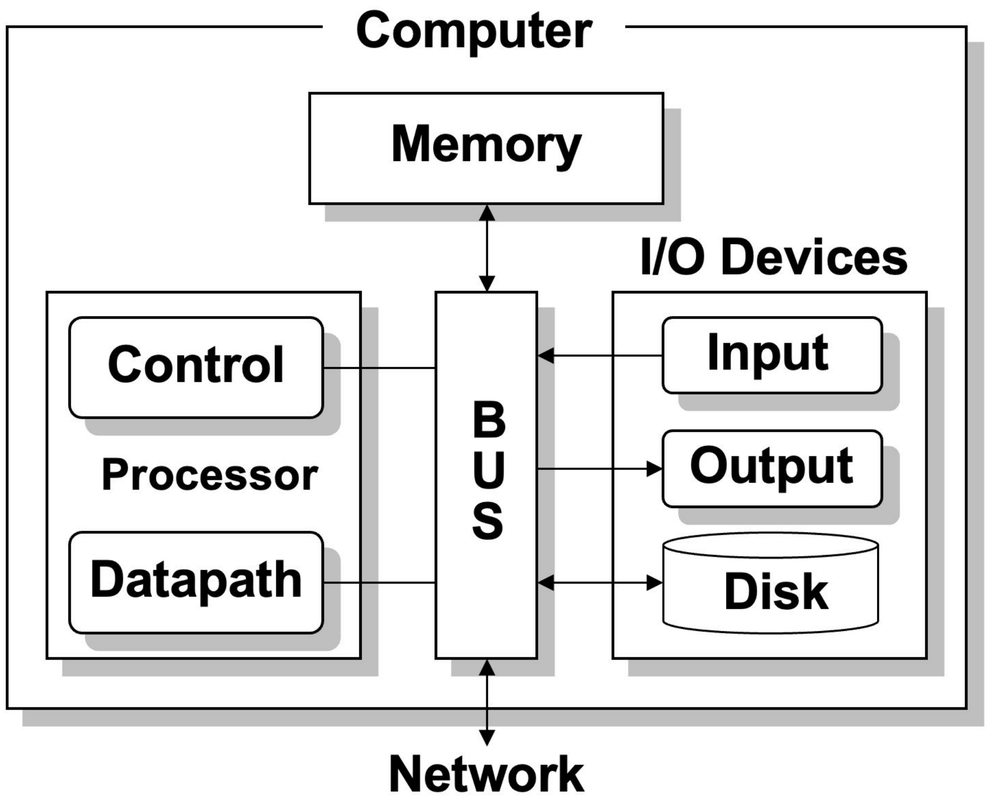
PART B

Memory

The memory system and how data is organized.

Components of a Computer System

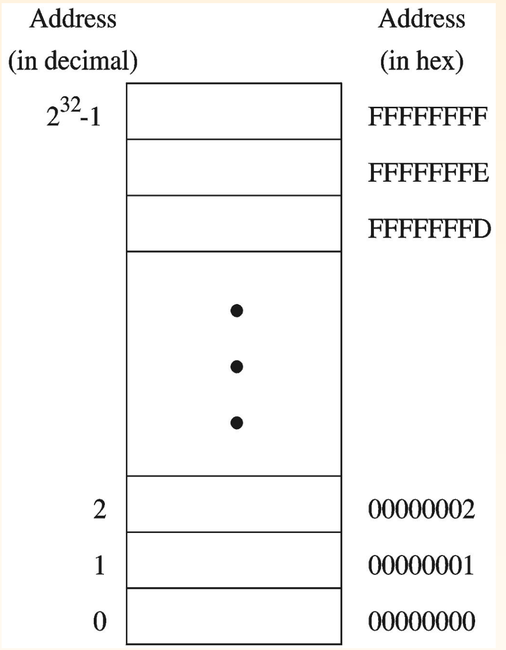
- Processor
  - Datapath
  - Control
- **Memory** & Storage
  - Main **Memory**
  - Disk Storage
- Input devices
- Output devices
- Bus: Interconnects the processor to the **memory** and the I/O
- Network: newly added component for communication



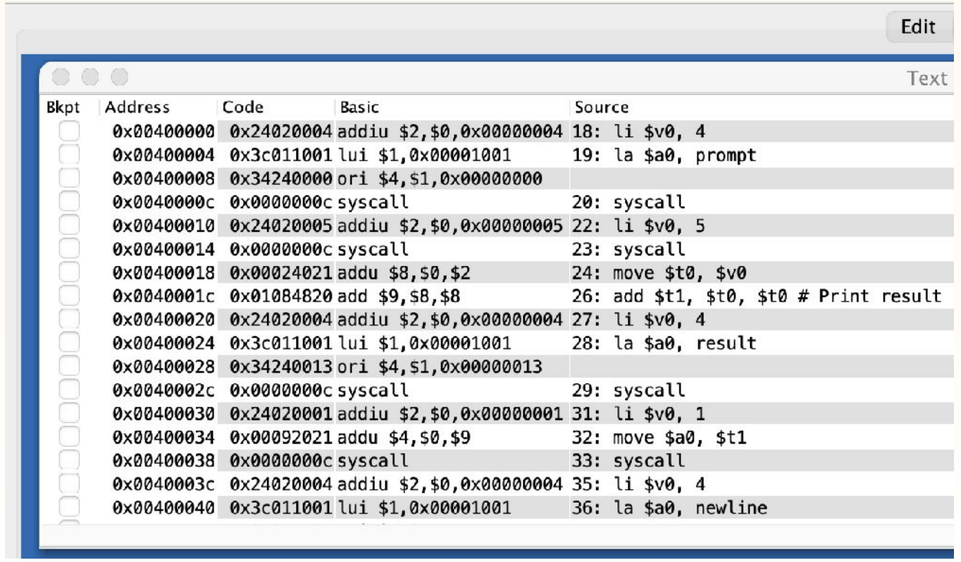
Memory







- Ordered sequence of bytes:
  - the sequence number is called the **memory address**.
- Byte addressable **memory**:
  - each byte has a unique address,
  - supported by almost all processors
- Physical address space:
  - determined by the address bus width,
    - Pentium has a 32-bit address bus
      - Physical address space = 4GB = 2^{32} bytes.
    - Itanium with a 64-bit address bus can support up to 2^{64} bytes of physical address space.



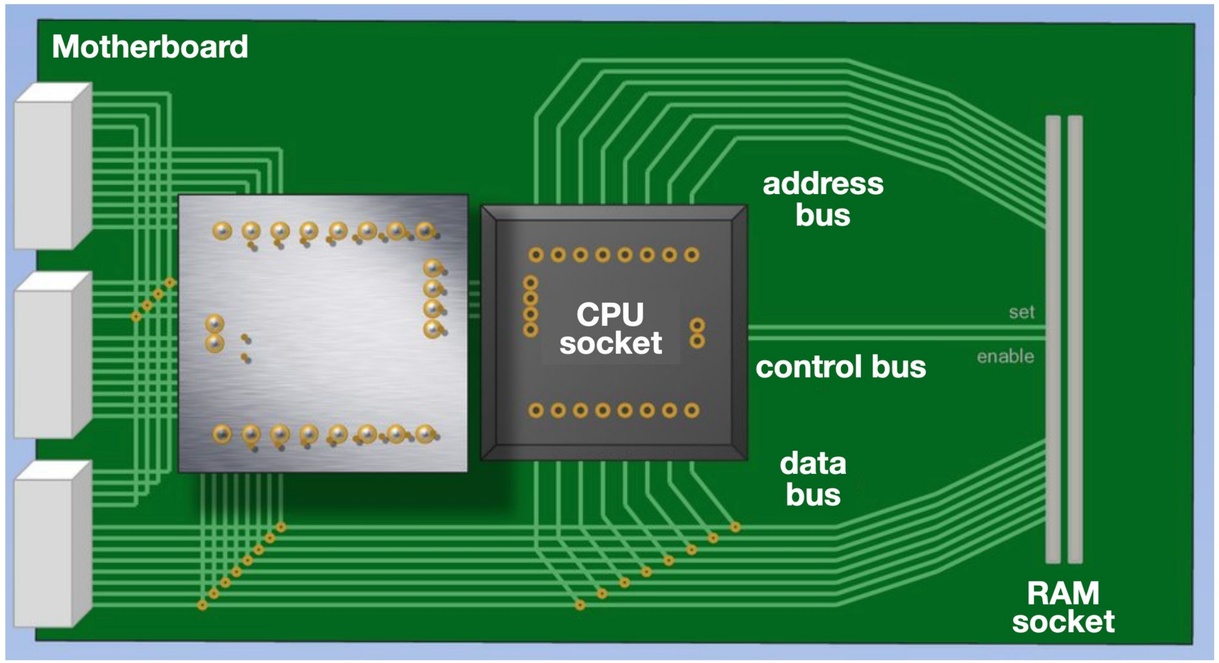
Bus

- A bus is …


15
Aldwych
Fleet Street
St. Paul's Cathedral
TOWER HILL
LONDON TRANSPORT
ALM 898B

Bus

- To **communicate** with each component in the system unit (*CPU, Main Memory, Devices, etc*), the motherboard component makes use of wires that are known as a 'bus.'
- A bus is a collection of **wires** or electronic circuits through which **information flows** from one component to another.



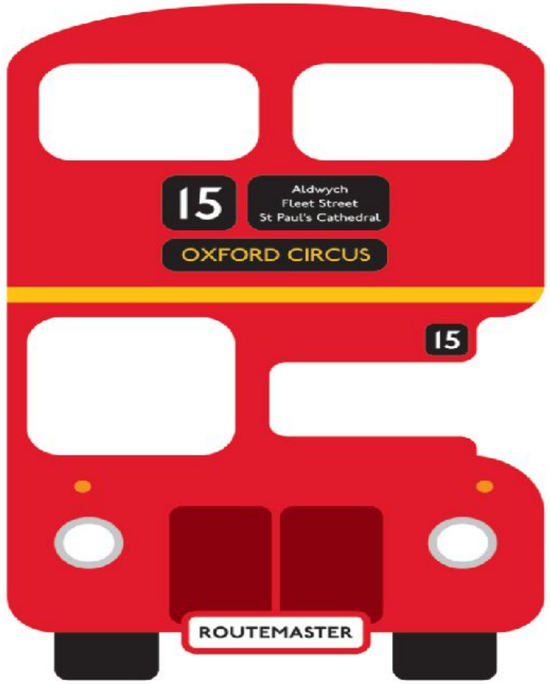
Bus Widths

- Buses come in different widths.



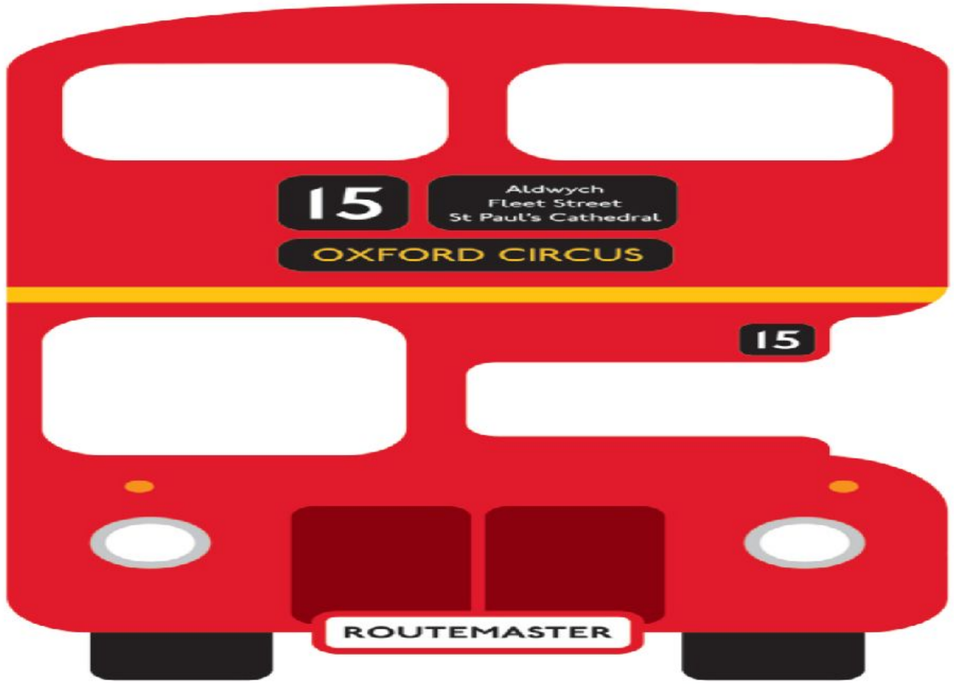
16-bit width

1010101010101010



32-bit width

10101010101010101010101010101010



64-bit width

1010101010101010101010101010101010101010101010101010101010101010

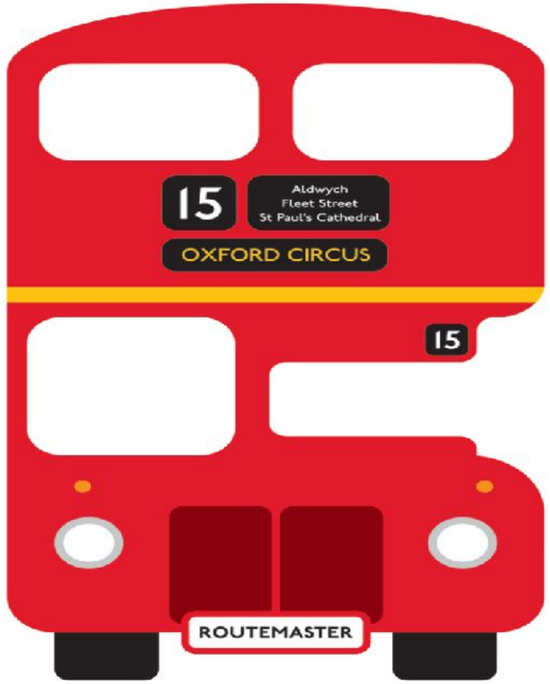
Bus Widths

- A 64-bit bus can naturally carry twice the data of a 32-bit bus per clock cycle.



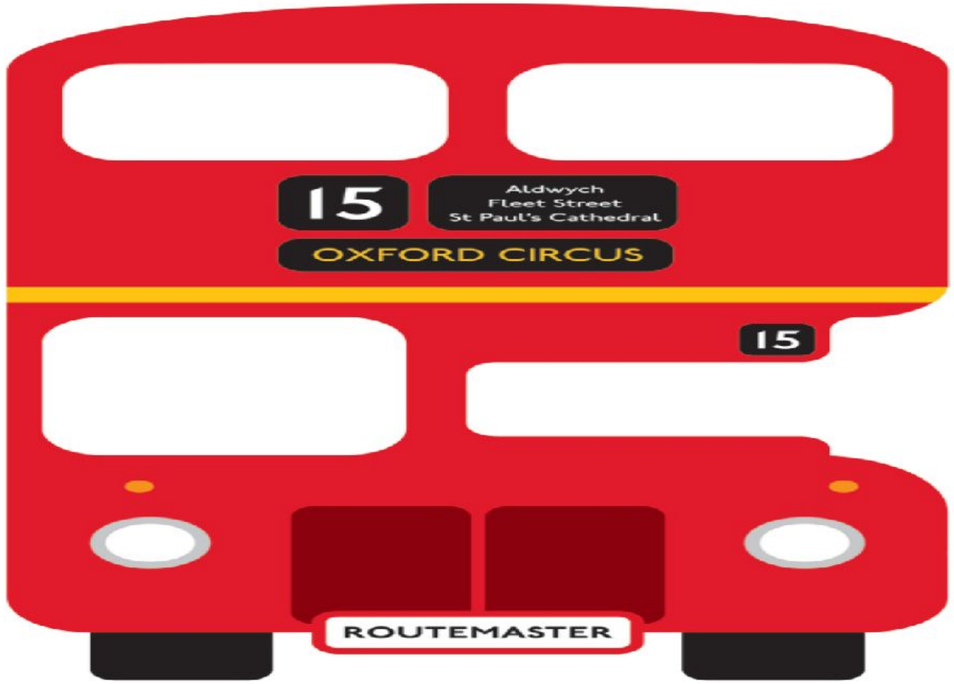
16-bit width

1010101010101010



32-bit width

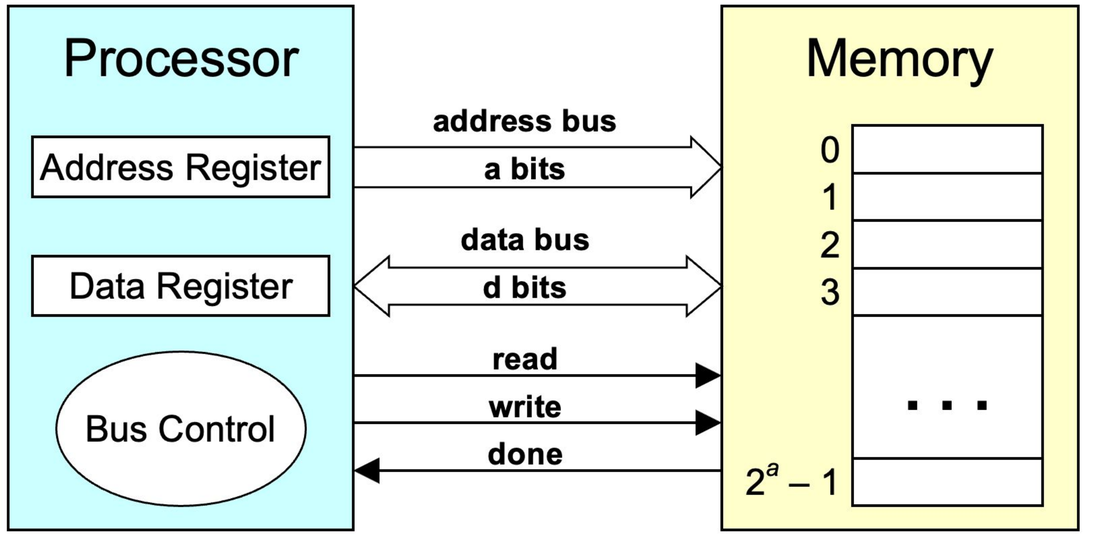
10101010101010101010101010101010



64-bit width

1010101010101010101010101010101010101010101010101010101010101010

Bus: three types of bus pathways



There are three primary types of pathways:

- **Address Bus**
  - **This carries Memory Addresses**
  - For ***a*** bits of memory, it can address ***2^a*** locations.

- **Data Bus: bi-directional bus**
  - **Data** can be transferred in both directions. ***But it has to wait. 1-way at a time.***
  - **Reading from or writing to memory / devices.**

- **Control Bus - Manage communication processes, such as main memory read/write:**
  - **Signals** control transfer of data
  - Read request
  - Write request
  - Done transfer

Bus: Example of three types of bus pathways

Data Bus ↔

- CPU ↔ Memory
- CPU ↔ I/O
- Memory ↔ I/O

Control Bus →

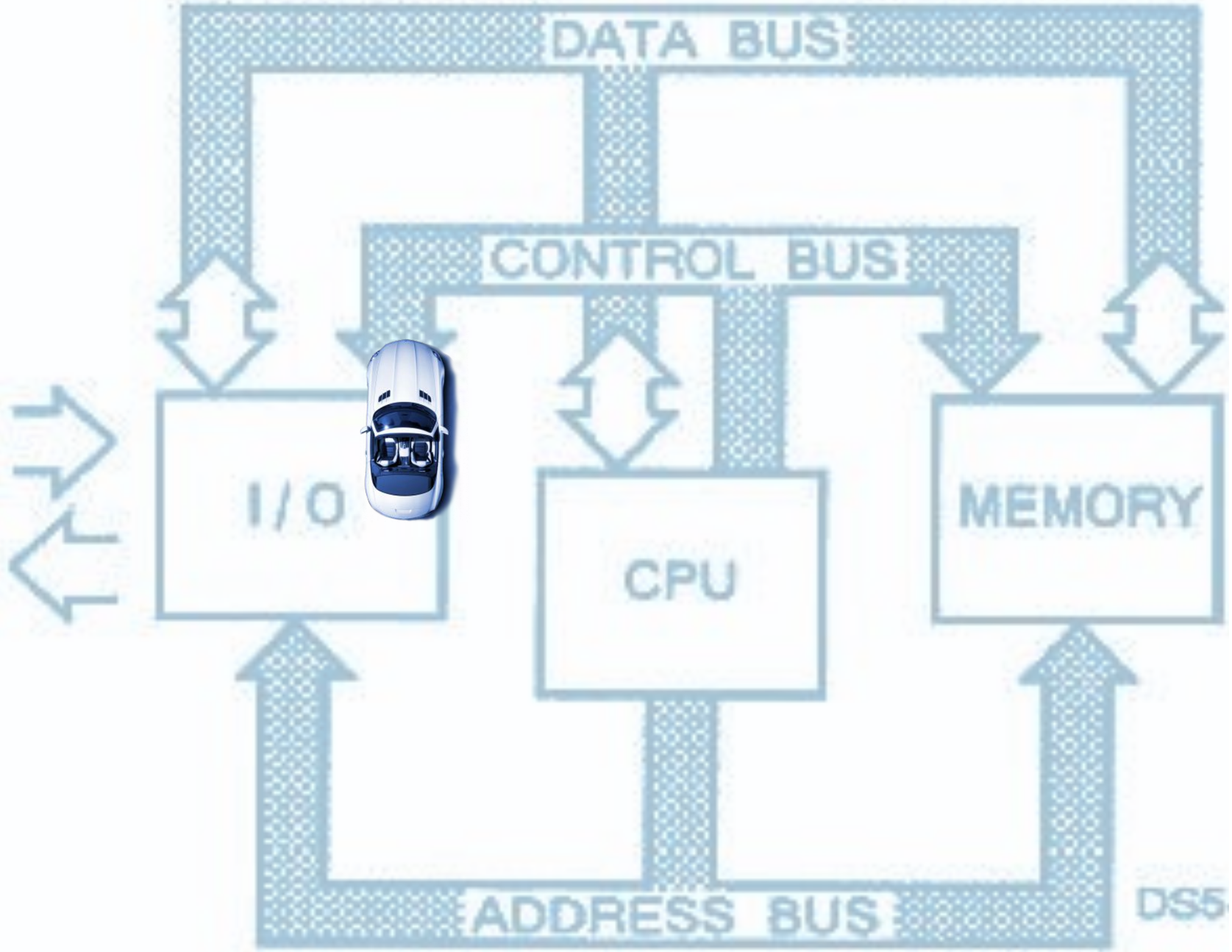
- CPU → Memory
- CPU → I/O
- Memory / I/O → CPU

Address Bus →

- CPU → Memory
- CPU → I/O

External I/O →

- External Devices → I/O
- I/O → External Devices



Types of Memory: RAM

- RAM (Random Access **Memory**) and ROM (Read-Only Memory) are two types of computer memory that serve different purposes:

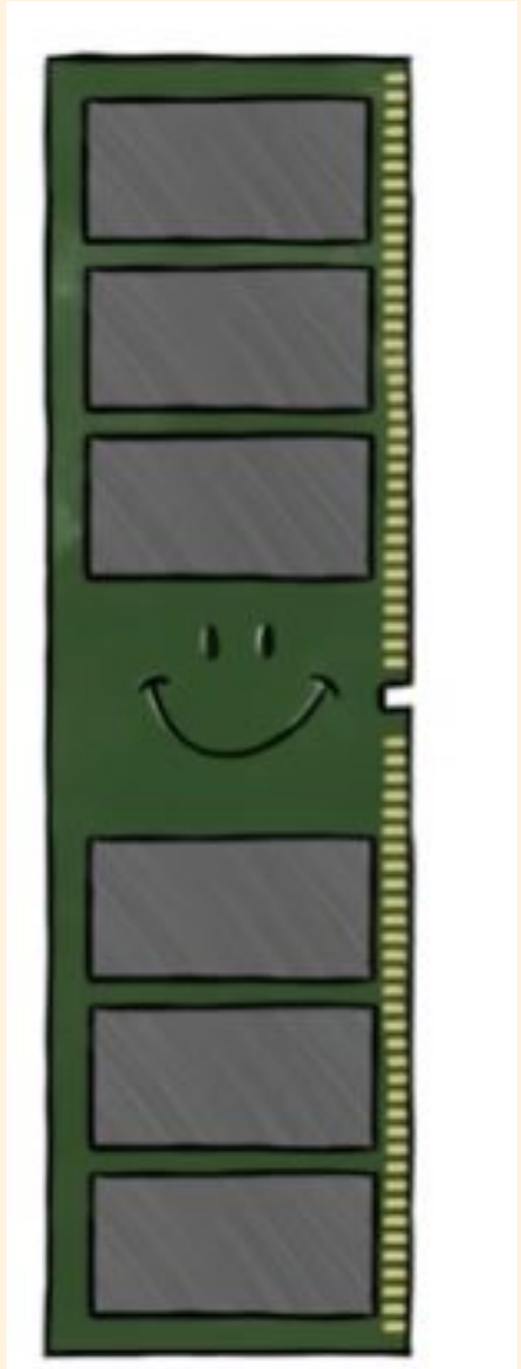
- RAM
  - Volatile **Memory**: Loses all data when the device is powered off.
  - Used For: Temporarily storing data that is being actively used or processed by the **CPU**.
  - Speed: Fast, allows quick read and write.
  - Example:
    - When you open apps or files, they load into RAM for faster access.

Types of Memory: ROM

- ROM
  - Non-Volatile **Memory**: Retains data even when the device is powered off.
  - Used For: Permanently storing firmware or system instructions needed for booting.
  - Speed: Slower than RAM, and usually read-only or hard to modify.
  - Example:
    - BIOS or UEFI firmware in computers.
      - UEFI (Unified Extensible Firmware Interface) is a modern, secure, and feature-rich firmware that serves as the interface between a computer's hardware and its operating system, replacing the older BIOS (Basic Input/Output System).
      - It enables a faster boot process, supports larger disk drives through disk partitioning, and provides enhanced security features like Secure Boot to prevent malware.

RAM

- RAM, Random access **memory**, is a temporary storage location where data can be retrieved or rewritten in any order to support the realtime working of computer and mobile applications.
    - Without the ability to store and retrieve data fast, none of the applications or other functions would work properly.

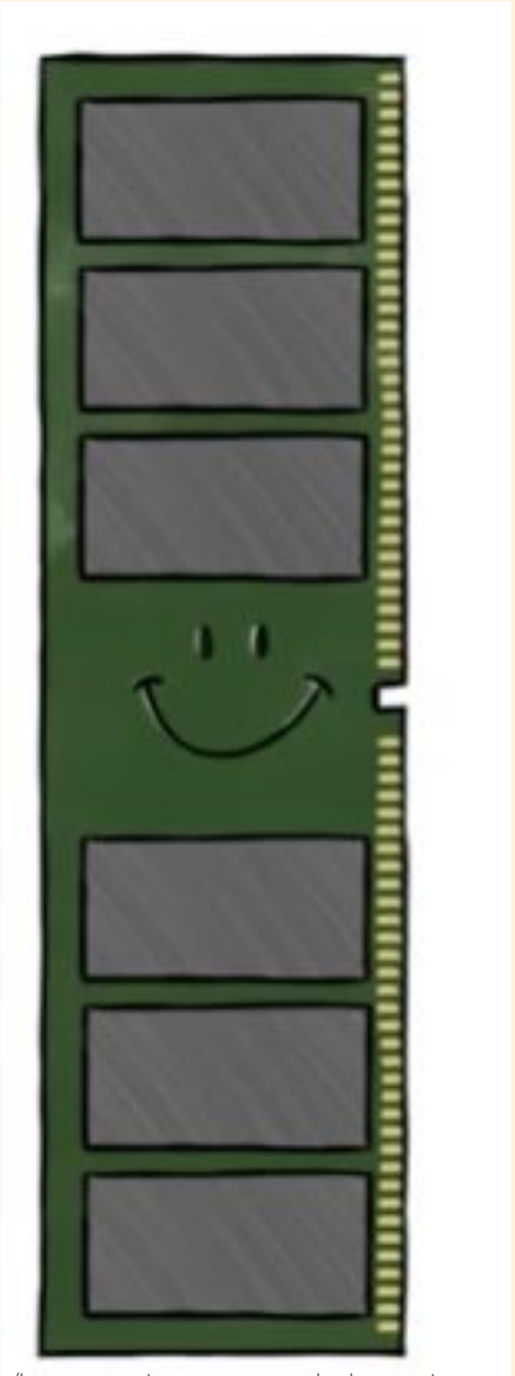


Video Image Source: https://ed.ted.com/lessons/how-computer-memory-works-kanawat-senanan

RAM

- RAM maintains data readily available so that the central processor unit (**CPU**) may locate it without requiring it to access long-term storage to perform urgent processing duties.

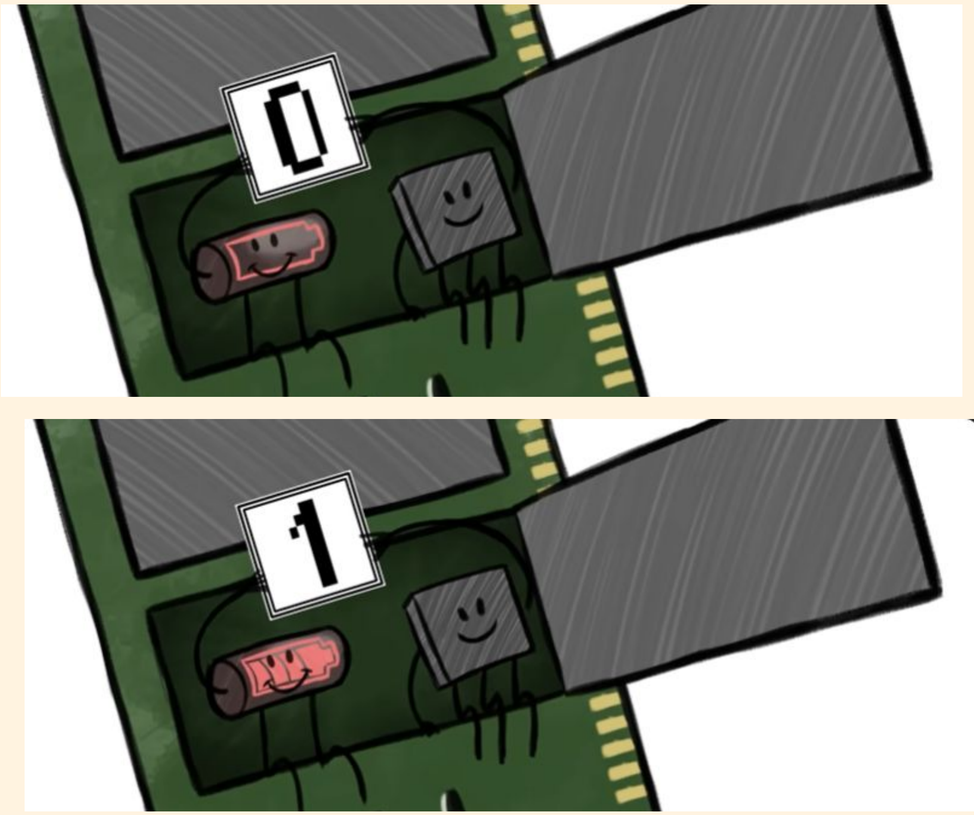
- RAM is included in all computing devices, including
  - desktop computers (operating on Windows, MacOS, and Linux),
  - tablets and
  - smartphones (running on Android or iOS), and
  - even internet of things (IoT) devices (like a smart TV).



Video Image Source: https://ed.ted.com/lessons/how-computer-memory-works-kanawat-senanan

DRAM

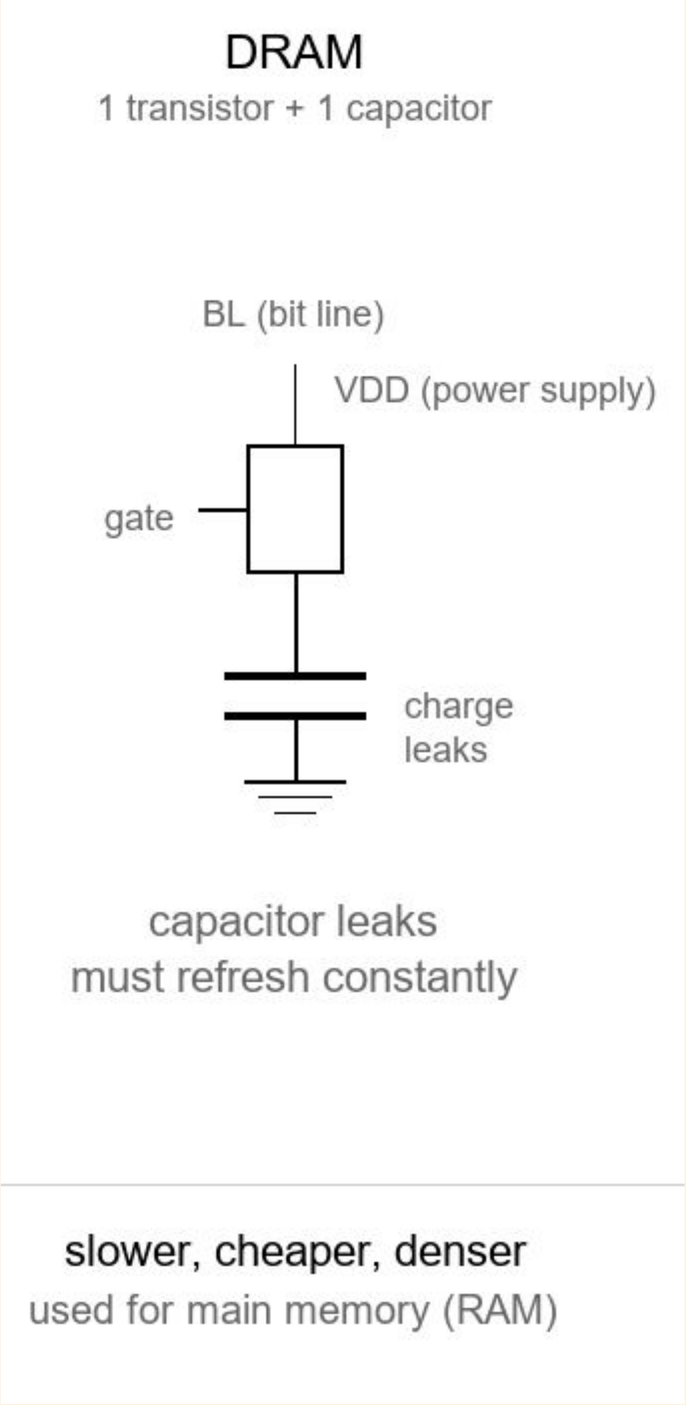
- DRAM (Dynamic RAM) is a specific, common, and inexpensive type of RAM.
  - It is called “dynamic” because it stores each bit of data in a tiny capacitor that must be constantly refreshed with an electrical charge to prevent the data from leaking away.
    - 1-Transistor cell + trench capacitor
    - Dense but slow, must be refreshed
    - Typical choice for main **memory**



Video Image Source: https://ed.ted.com/lessons/how-computer-memory-works-kanawat-senanan

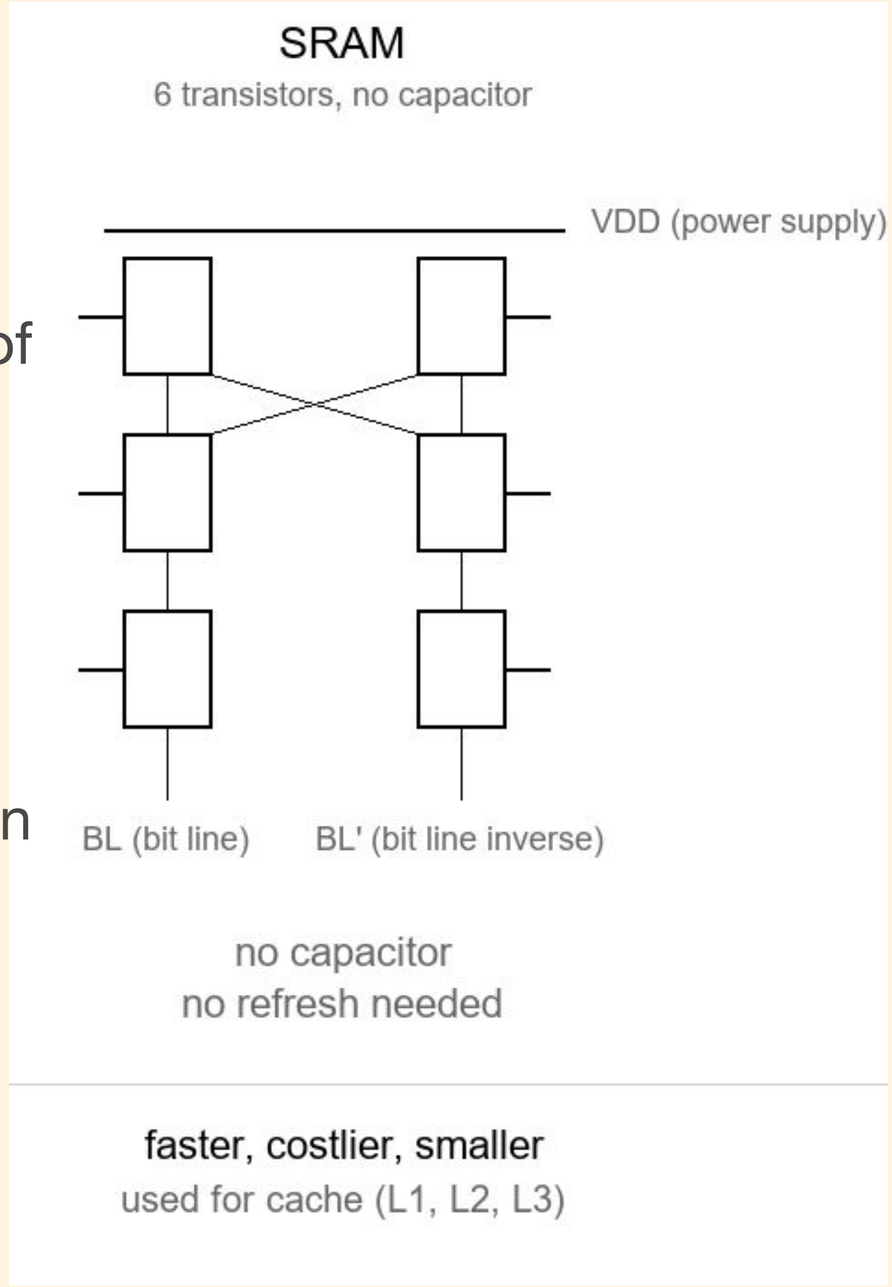
DRAM

- DRAM (Dynamic RAM)
  - DRAM is often **utilized as the primary memory** in computers.
  - A data byte is retained in every DRAM **memory** cell's capacitor, consisting of transistors and a capacitor in an integrated circuit.
  - To maintain data, DRAM must be updated every few milliseconds.



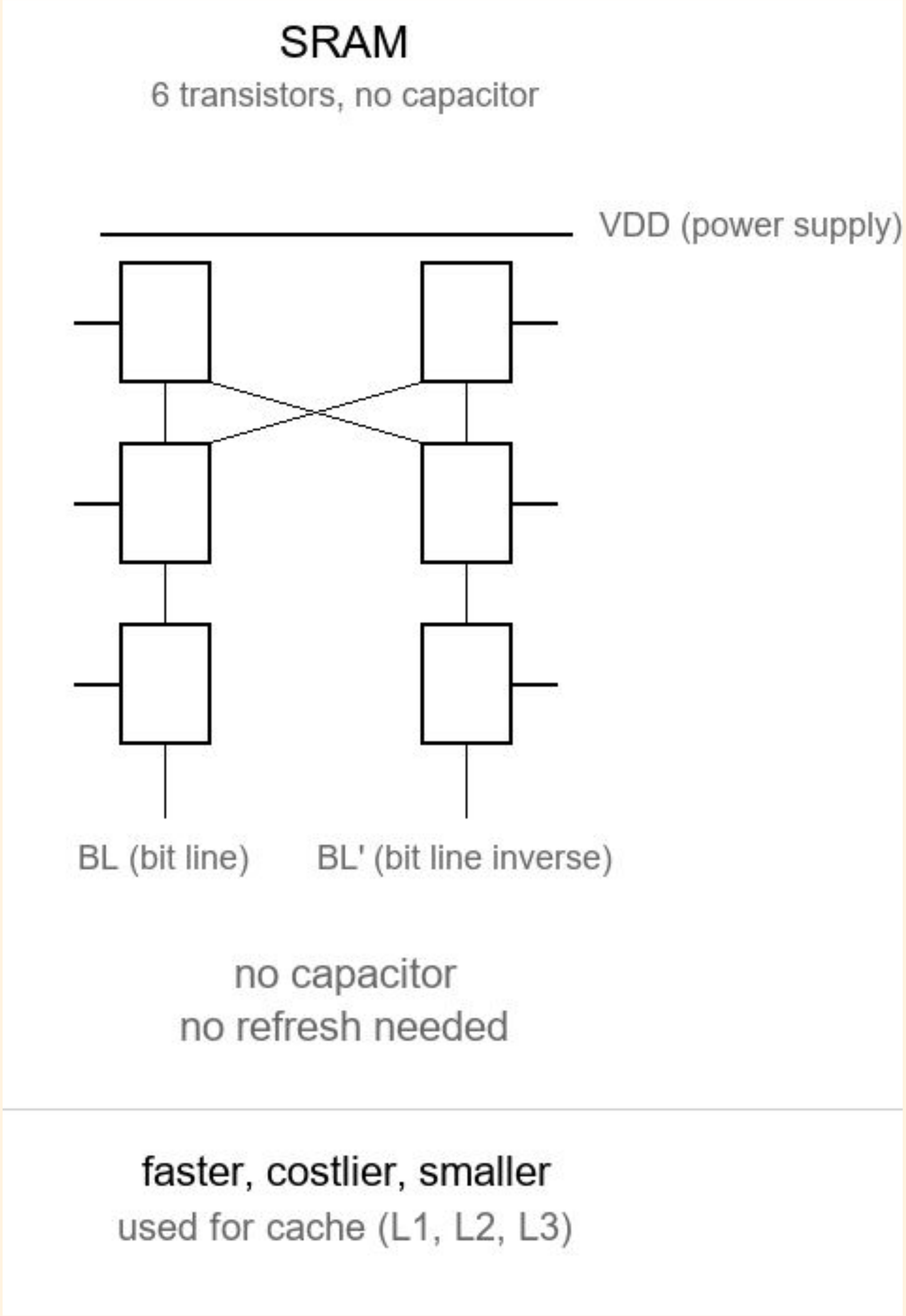
SRAM

- **SRAM (Static RAM)** is the other main type of RAM.
  - Unlike DRAM, it does not need to be constantly refreshed, making it faster and more expensive.
  - SRAM is typically used for smaller, **high-speed cache memory** directly on the **CPU**.
    - 6-Transistor cell, faster but less dense than DRAM
    - Typical choice for **cache memory**

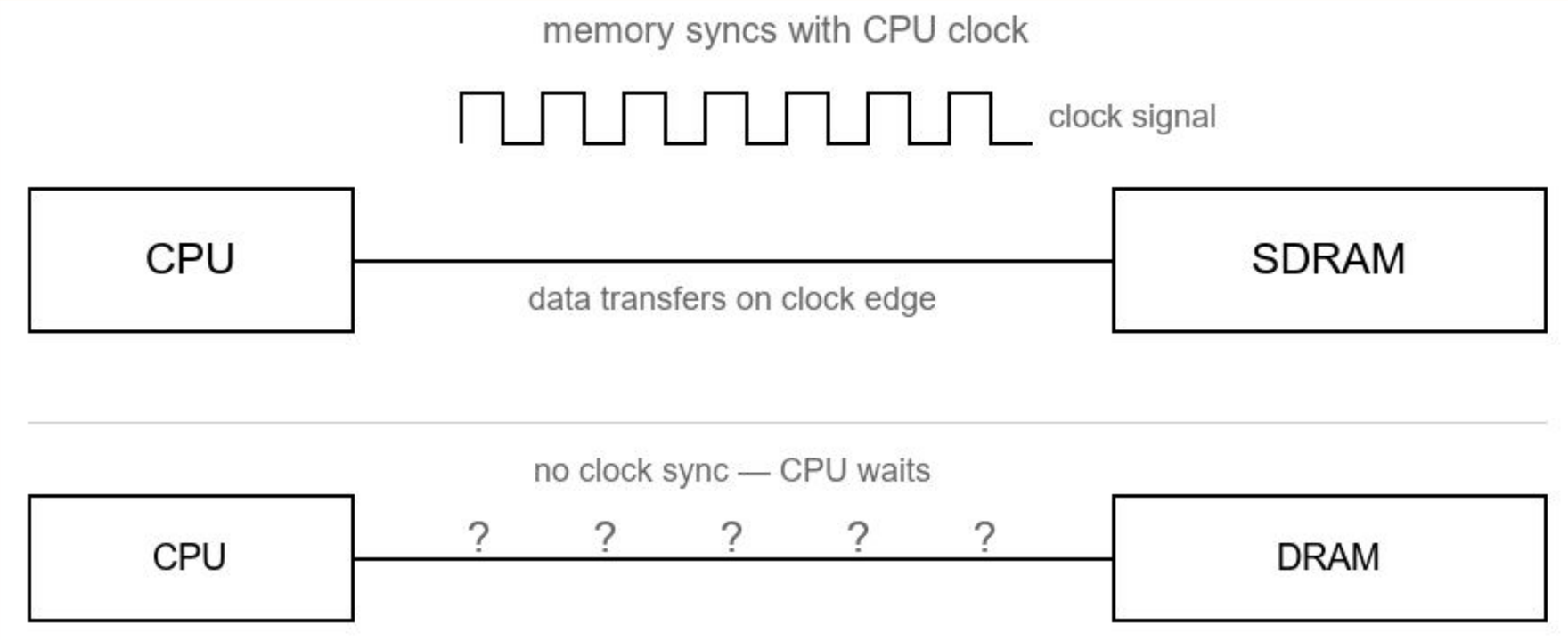


SRAM

- SRAM (Static RAM)
  - For each **memory** cell, static random-access memory requires many transistors, generally four to six.
  - Each cell does not contain a capacitor.
    - It is typically used for caching.
  - SRAM **preserves data in memory if power is available**, in contrast to dynamic RAM (DRAM), which must be frequently updated.
  - As a result, SRAM is quicker but costlier, rendering DRAM the more common kind of **memory** in computers.



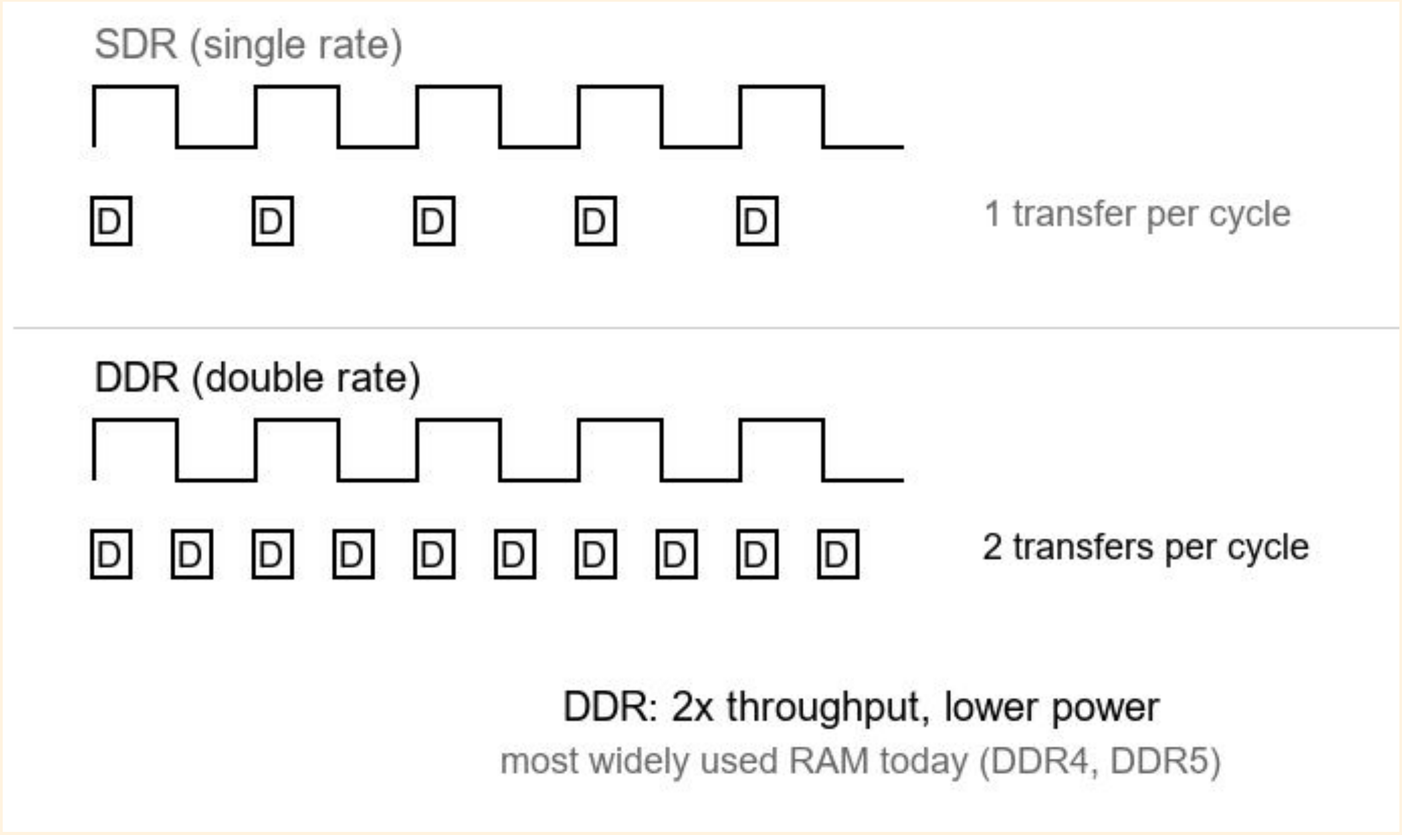
SDRAM (Synchronous DRAM)



- This form of RAM **synchronizes memory rates** with the clock speed of the central processor unit (**CPU**).
- Consequently, the **memory controller is aware of the precise clock cycle at which the data packet will be available**.
- This enables the **CPU** to execute more instructions per unit of time.
- SDRAM often transports data at up to 133 MHz, significantly faster than DRAM.

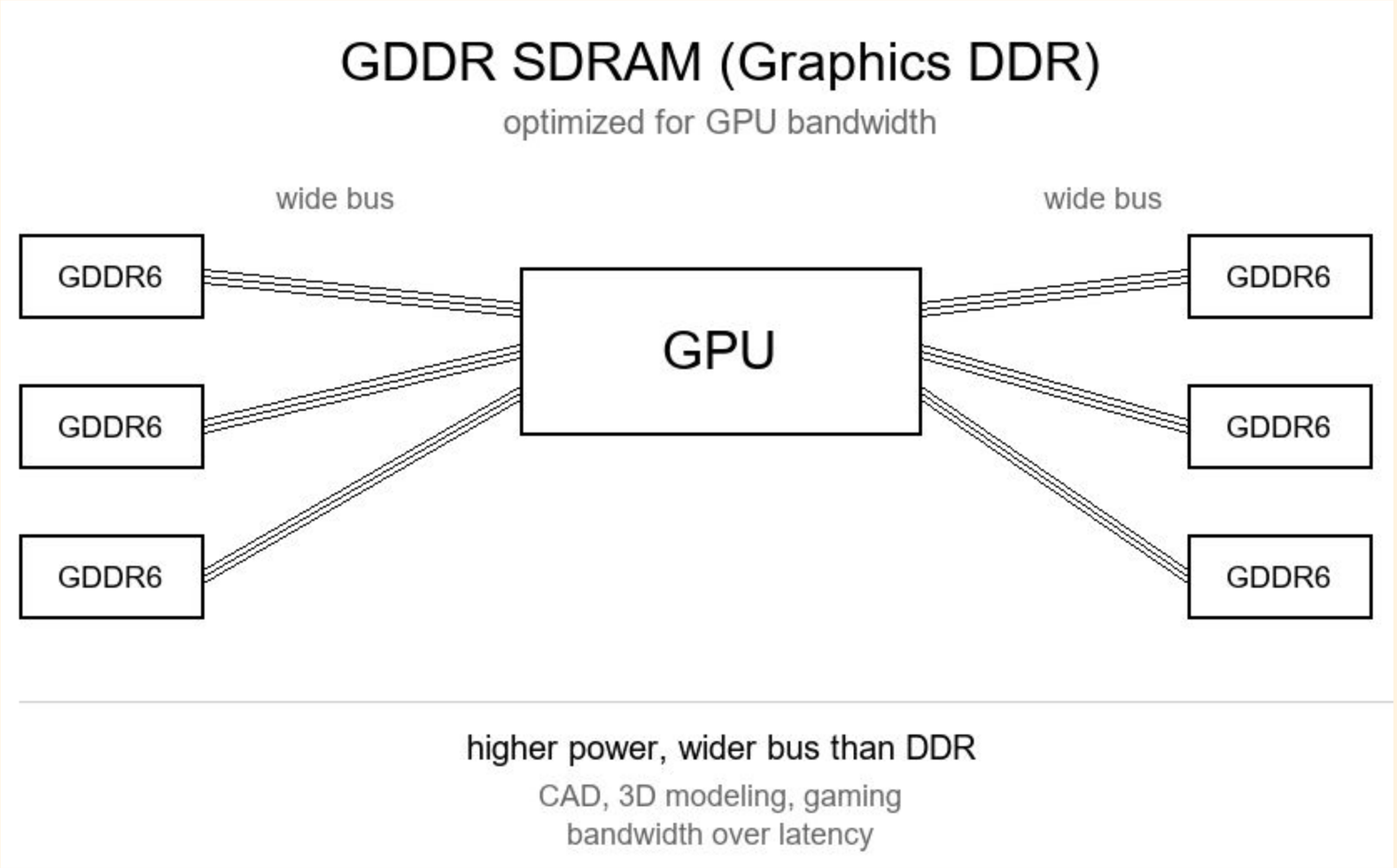
DDR SDRAM

- DDR SDRAM (Double Data Rate SDRAM)
  - DDR SRAM was invented around 2000.
  - This performed two data transfers in a singular clock cycle at the commencement and finish.
  - DDR SDRAM has been an increase in data transfer rates and a decrease in power consumption.
  - DDR is still the most widely used RAM in computers today.

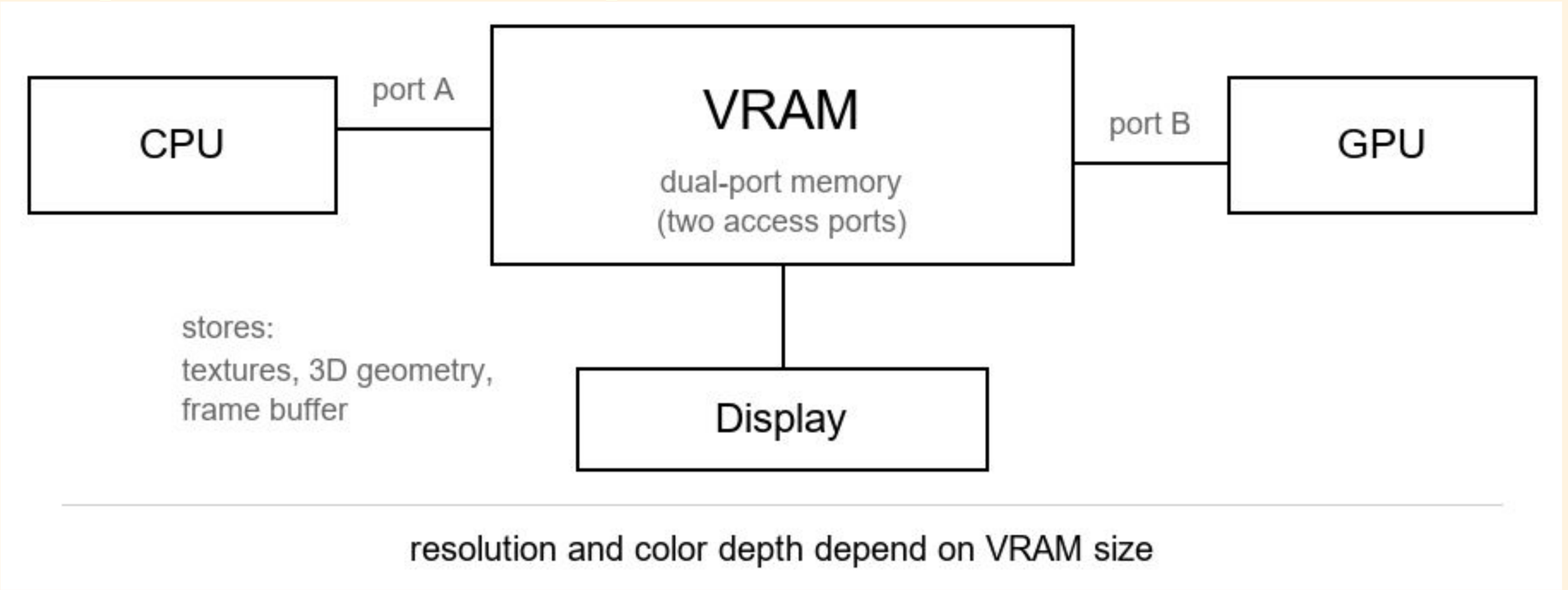


GDDR SDRAM

- GDDR SDRAM (Graphics Double Data Rate SDRAM)
  - GDDR SDRAM is utilized in video and graphics devices.
  - This architecture allows data to be transferred at several places within a **CPU** clock cycle.
  - It operates at greater power and has less stringent timing requirements than DDR SDRAM.
  - GDDR can **enable the speed, memory**, and bandwidth levels needed for the GPU performance required for graphics-intensive tasks such as computer-aided design (CAD) or 3D modeling.
  - The most **recent iteration of graphics memory** is GDDR6.



VRAM (Video RAM)



- VRAM is used only by video adapters and 3D accelerators.
- It often has two separate access ports instead of one (multiport), enabling the **CPU and the graphics processor** to concurrently access the RAM.

- A computer display's resolution and color depth is determined by the **quantity of video memory** (VRAM).
- Additionally, VRAM is utilized to store graphics-specific data like 3D geometry data or texture maps.



Key Points: Computer Memory Systems

- DRAM (capacitor, needs refresh) vs SRAM (transistors, faster, no refresh)
  - DDR SDRAM doubles data rate;
  - GDDR optimized for graphics
- Bus system connects CPU, memory, and I/O components
  - Address bus specifies location;
  - data bus carries values

PART B

Memory Hierarchy in more depth

The memory system: hierarchy, cache and how data is organized.

Memory Hierarchy

- Widening speed gap between **CPU** and main **memory**:
  - processor operation takes less than 1 ns,
  - main **memory** requires more than 50 ns to access.
- Each instruction involves at least one **memory** access:
  - one **memory** access to fetch the instruction,
  - a second **memory** access for load and store instructions.
- **Memory** bandwidth limits the instruction execution rate.
- **Cache memory** can help **bridge the CPU-memory gap**.
- **Cache memory** is small in size but fast.

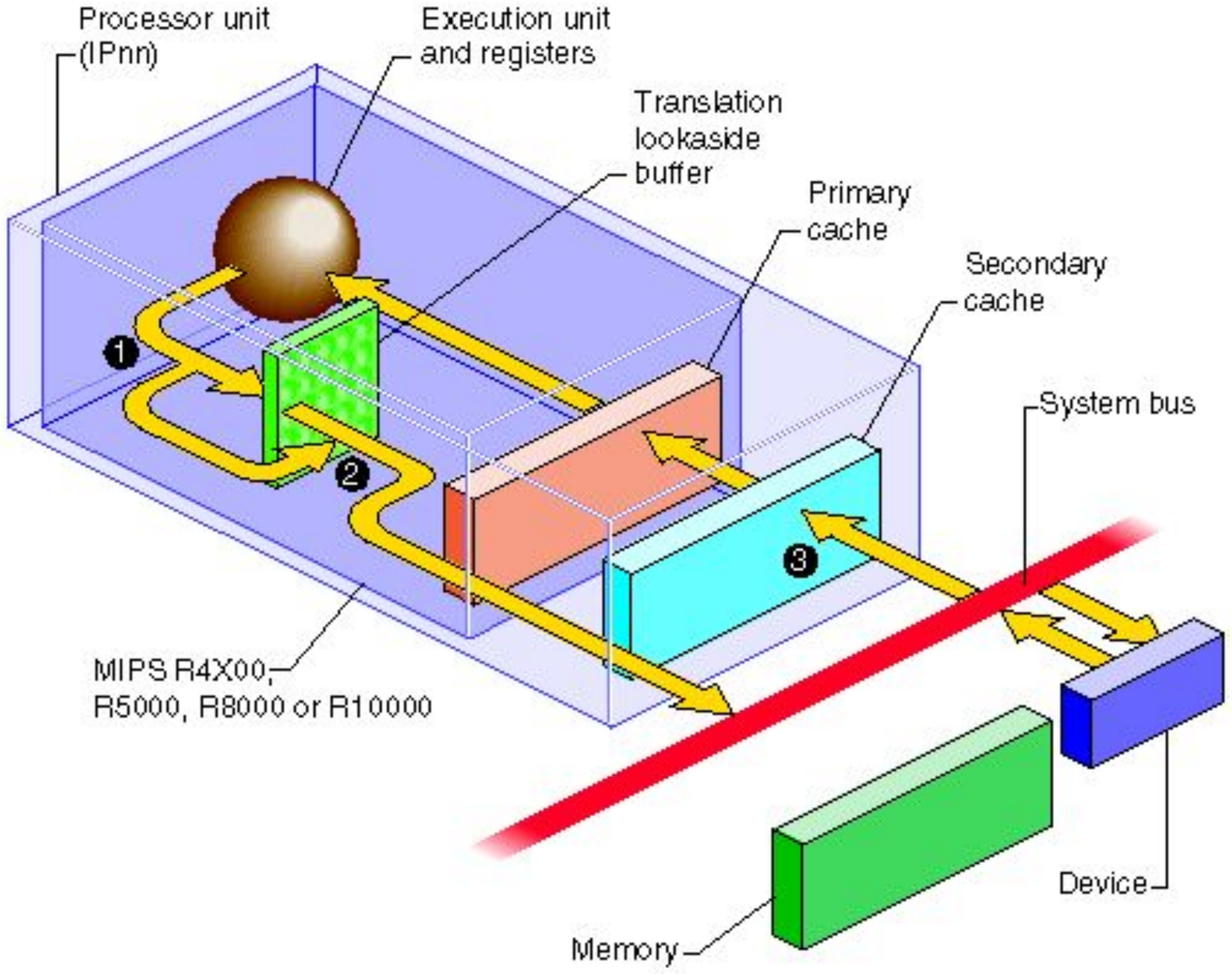
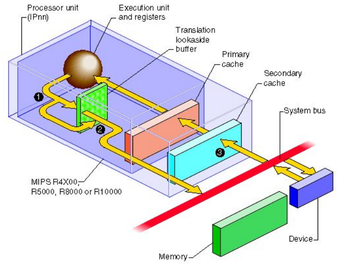
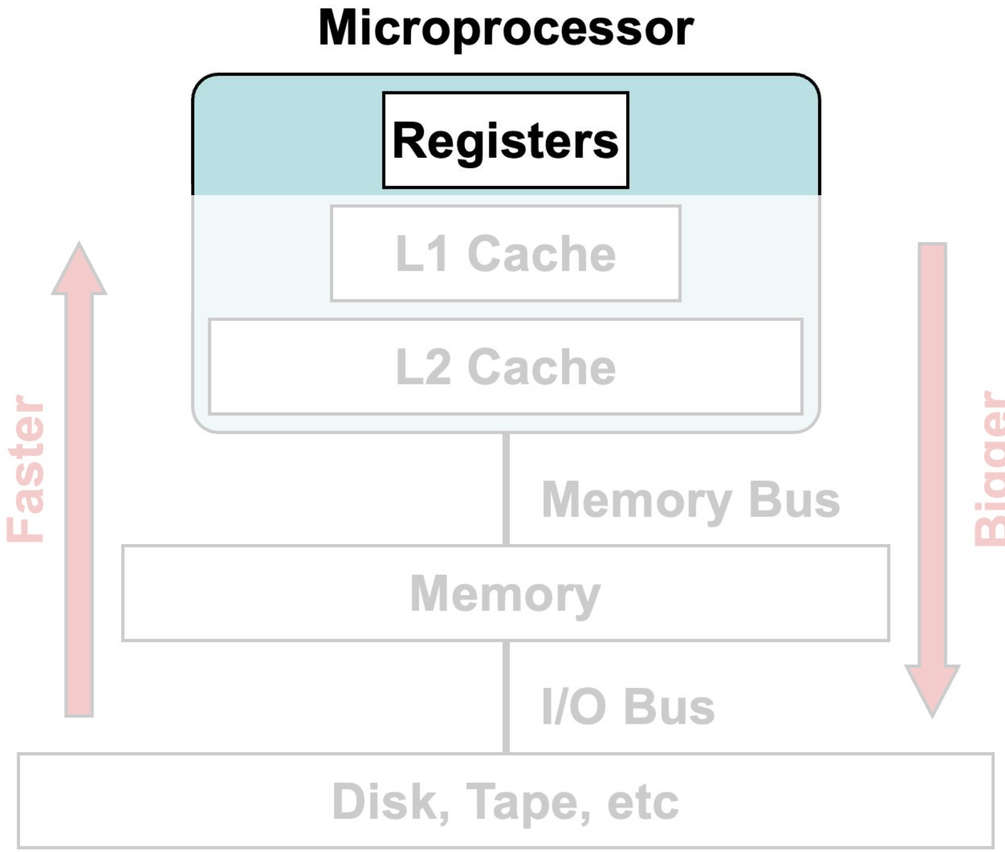


Image Source:
https://tqd1.physik.uni-freiburg.de/library/SGI_bookshelves/SGI_Developer/books/DevDriver_PG/sgi_html/ch01.html

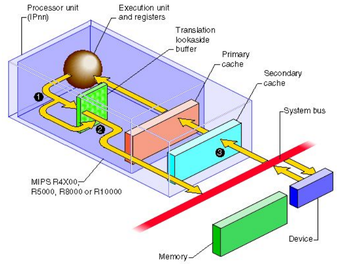
Memory Hierarchy



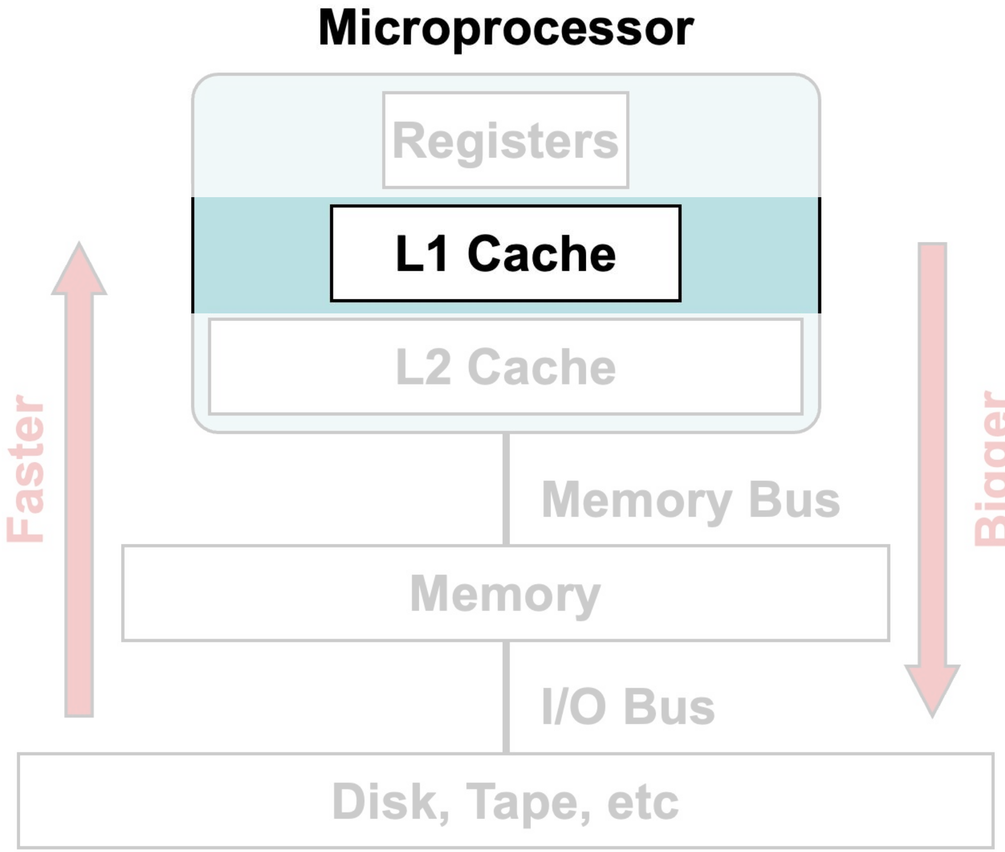
- Registers are at the top of the hierarchy
  - Located inside the **CPU** core itself.
  - Typical size < 1 KB; very small, usually less than 1 KB, for example, a **CPU** might have a set of 16 general-purpose registers, each 64 bits (8 bytes) wide, totaling 128 bytes.
  - Access time < 0.5 ns.; the fastest, typically less than 0.5 ns (nanoseconds).
    - The access time is often measured within a single **CPU** clock cycle.



Memory Hierarchy



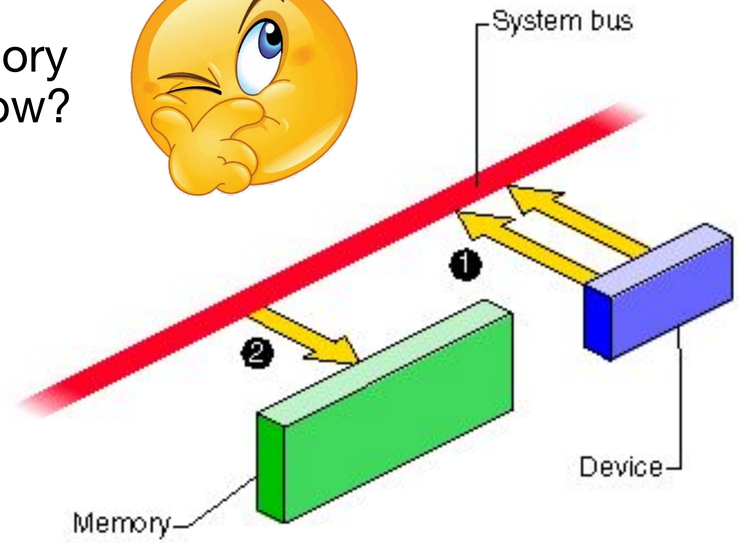
- Level 1 (L1) caches; for a modern high-performance **CPU**, like your PCs and smartphones:
  - Typical Size: 32 - 128 KB per core.
    - It's almost always split into two parts:
      - L1 Instruction **Cache** (L1i): ~32-64 KB
      - L1 Data **Cache** (L1d): ~32-64 KB
  - Access Time: ~1 - 2 ns., around 4 clock cycles on a modern **CPU**.
  - Location: Physically located on the same silicon die **as the CPU core**, making it the closest and fastest **cache**.



Memory Hierarchy

Why is the Memory read/write so slow?





- Overall capabilities among RAM and caches:

Memory Type	Typical Size	Access Time (nanoseconds)	Access Time (CPU Cycles)
Registers	< 1 KB	**~0.1 - 0.5 ns**	1 cycle
L1 Cache	32 - 128 KB (per core)	**~1 - 2 ns**	~4 cycles
L2 Cache	256 KB - 1 MB (per core)	**~3 - 10 ns**	~10 - 20 cycles
L3 Cache	4 - 64 MB (shared)	**~10 - 20 ns**	~30 - 50 cycles
Main Memory (RAM)	8 - 64 GB	**~50 - 100 ns**	**~200+ cycles**

Image Source:
https://tqd1.physik.uni-freiburg.de/library/SGI_bookshelves/SGI_Developer/books/DevDriver_PG/sgi_html/ch01.html

Memory Hierarchy

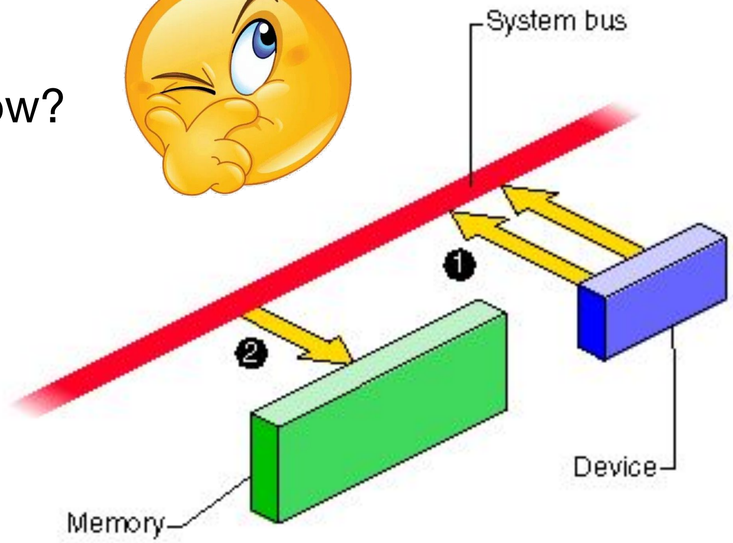
- Hard Disk Drives (HDDs), which use spinning magnetic platters, it's crucial to mention Solid State Drives (SSDs), which have a radically different performance profile:

Feature	Hard Disk Drive (HDD)	Solid State Drive (SSD)
Technology	**Moving parts: spinning magnetic platters** and a read/write head.	**No moving parts: flash memory chips** (like a USB drive).
Access Time	**Milliseconds (ms)**. Slower due to physical movement.	**Microseconds (µs)**. **~100x faster** than HDD. No seek time or latency.
Typical Access	**~8 - 12 ms**	**50 - 200 µs**
Primary Cost	Cheap per GB ($/GB)	More expensive per GB ($/GB)
Primary Use	Bulk storage: large archives, media libraries, backups.	Operating system, programs, games, frequently accessed files.

Memory Hierarchy

Why is the SDD read/write so slow?



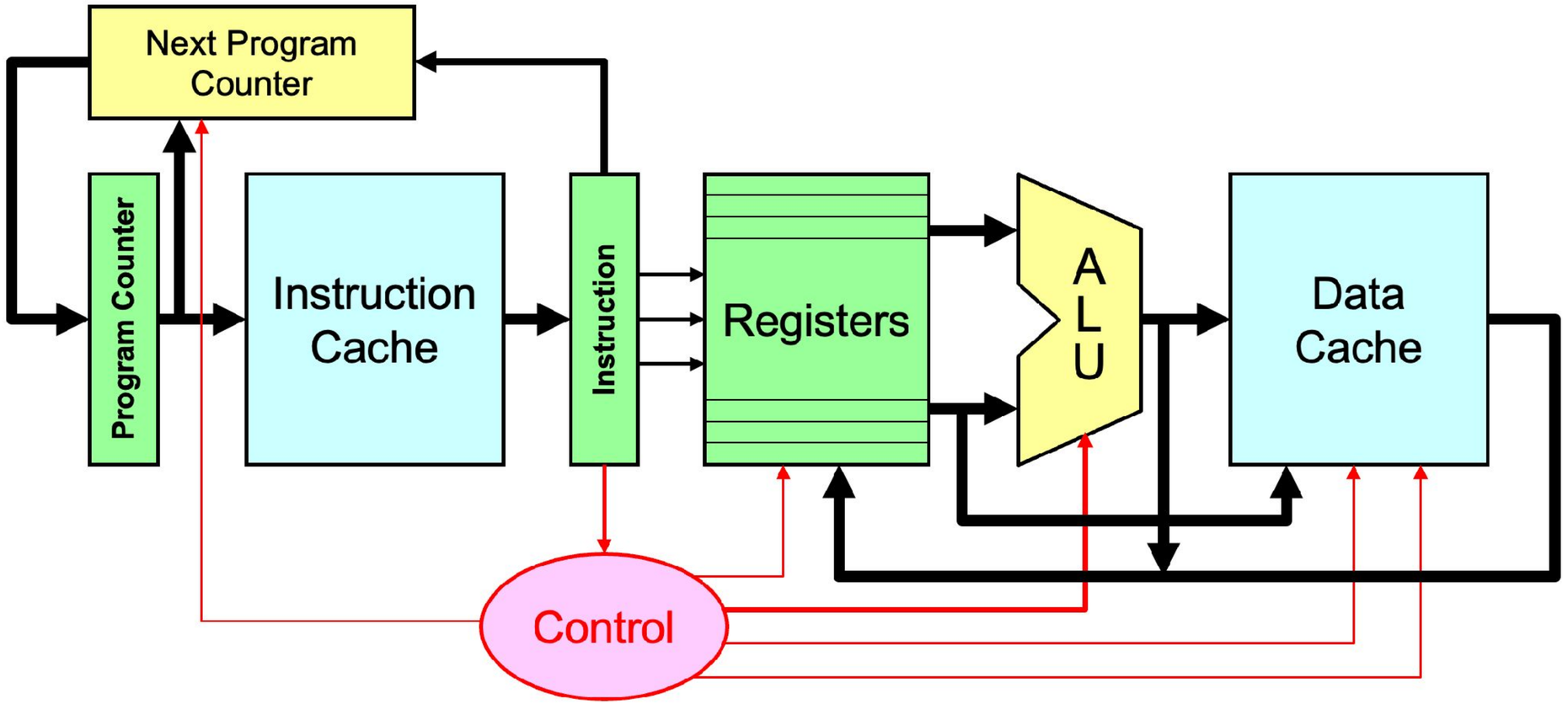


- In overall comparison of **memory** storage capabilities:

Memory Type	Typical Access Time	Relative Speed Analogy
CPU Register	0.5 ns	**1 second**
L1 Cache	~1 ns	**2 seconds**
RAM	~100 ns	**3.3 minutes**
SSD	~100,000 ns (100 µs)	**~2.8 days**
HDD	~10,000,000 ns (10 ms)	**~ ~11.5 months**

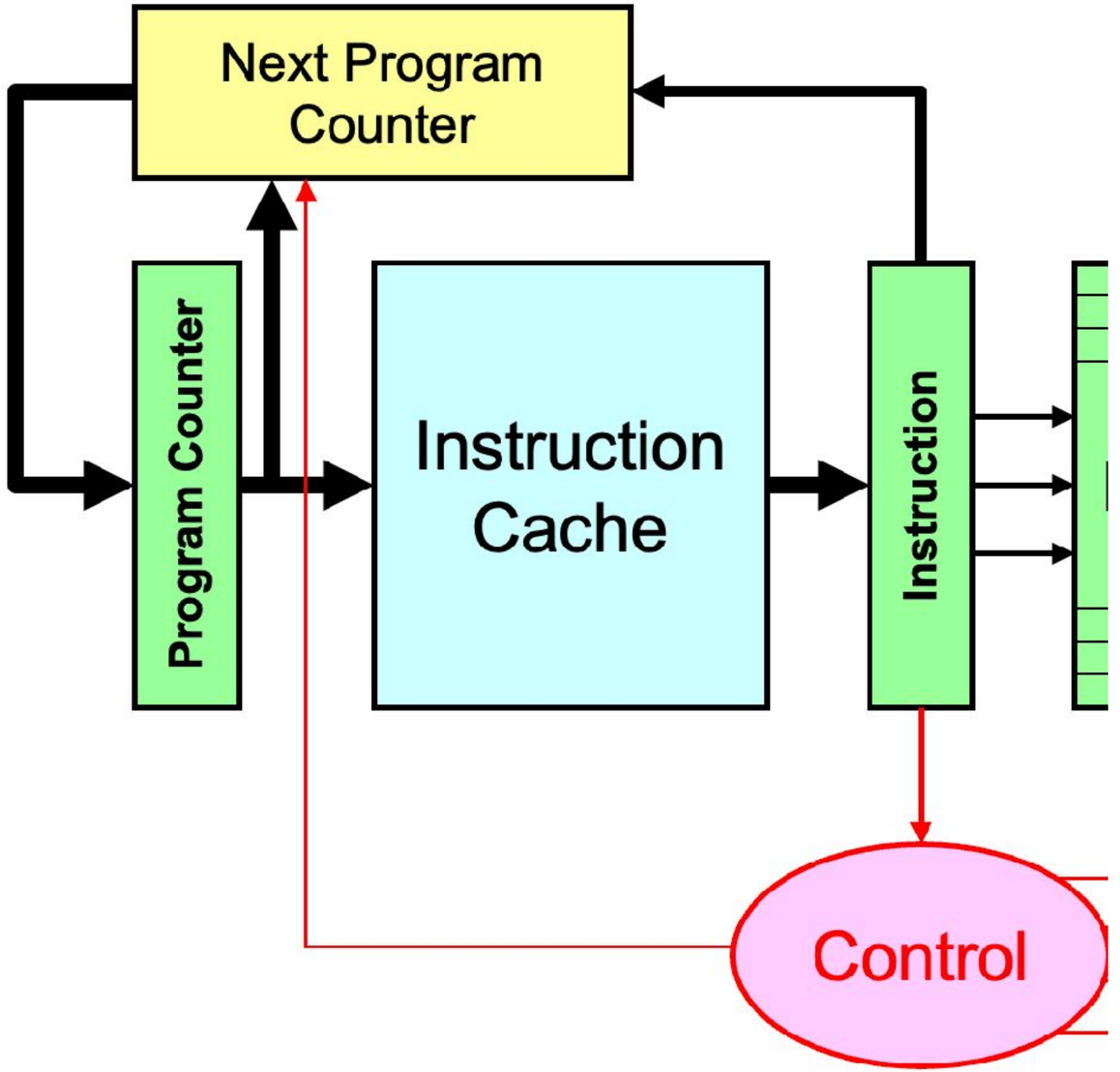
Processor Datapath & Control Signals

- Datapath: part of a processor that executes instructions.
- Control: generates control signals for each instruction.



Datapath Components

1. **Program Counter (PC)**
   - contains address of instruction to be fetched,
   - next Program Counter: computes address of next instruction.
1. **Instruction Register (IR)**
   - stores the fetched instruction.
2. **Instruction and Data Caches**
   - small and fast **memory** containing most recent instructions/data.



Datapath Components

2 Register File

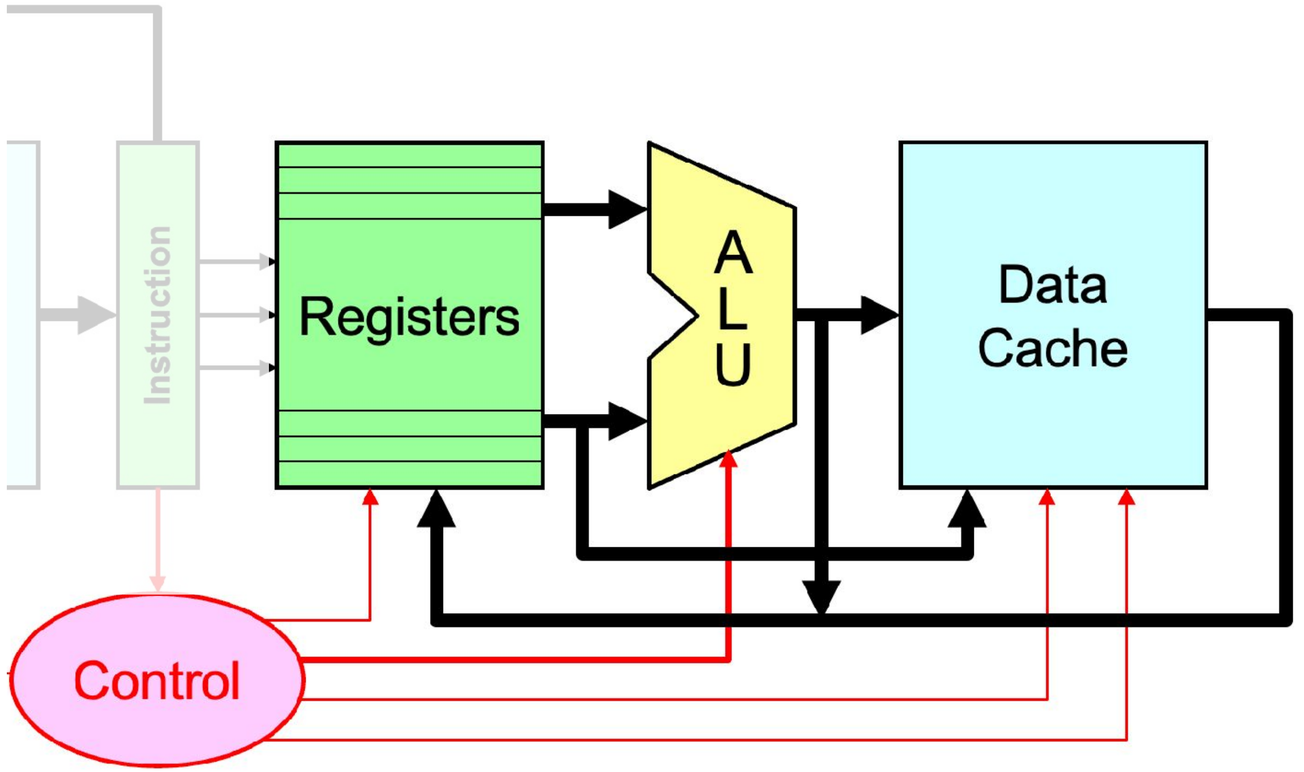
- general-purpose registers used for intermediate computations

3 ALU = Arithmetic and Logic Unit

- executes arithmetic and logic instructions.

? Buses

- used to wire and interconnect the various components.



Fetch-Execute Cycle

5-stage **Pipeline**



EXERCISE

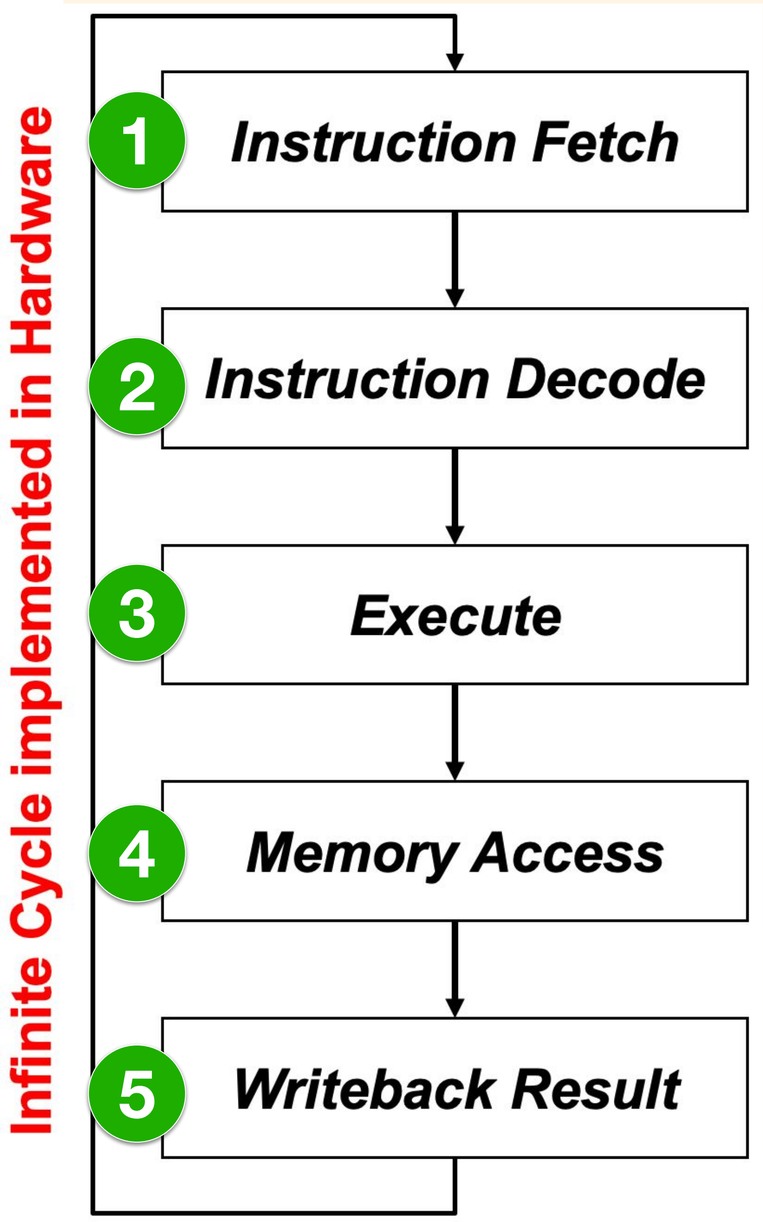
a Read or write **memory** (load/store)

b Fetch instruction
- Compute address of next instruction

c Generate control signals for instruction
- Read operands from registers

d Writeback result in a **register**

e Compute result value



Fetch-Execute Cycle

5-stage **Pipeline**



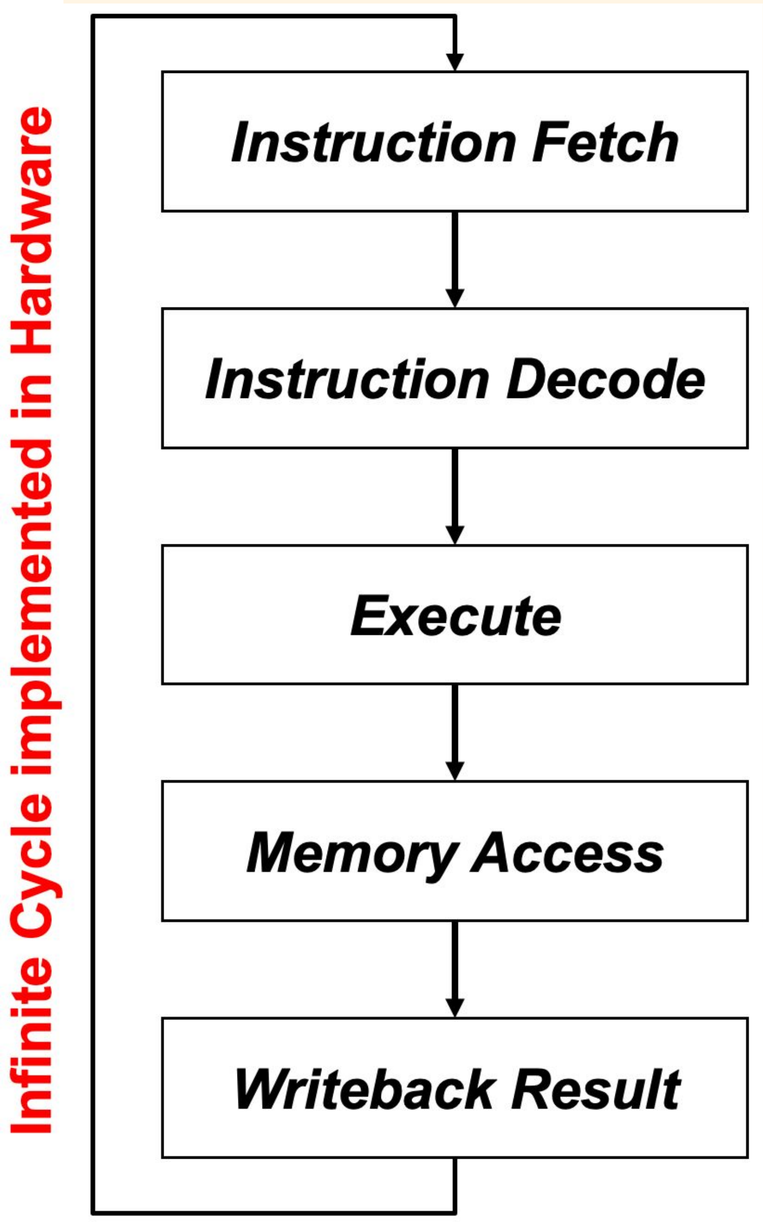
EXERCISE

A **Load Word** (**LW**) **instruction** (read from memory) produces its result at the end of the MEM (**Memory Access**) stage.

Why does this create a pipeline **Data Hazard** that even **data forwarding** cannot eliminate?



More about Load Word (lw) in a few minutes.





Key Points: Computer Memory Systems

- Memory hierarchy: Registers > Cache (L1/L2) > RAM > Storage
  - Each level trades capacity for speed;
  - CPU accesses fastest first
- Processor: Datapath executes instructions, Control generates signals
  - Program Counter (PC) tracks address of next instruction to fetch

PART C

MIPS Assembly with Cache and Memory

Bringing MIPS assembly together with cache and memory behaviour.

MIPS assembly

- In **MIPS assembly**, interaction with **memory** is governed by a strict hierarchy, where
  - the **CPU**'s registers are the fastest,
  - followed by one or more levels of **cache**, and
  - finally the much slower main **memory** (DRAM).

- **MIPS** is a load/store architecture,
  - Its **memory operations** are only handled by explicit **load** and **store instructions**,
  - *Its arithmetic and logic operations work only on registers. (as we know already)*

- A 32-bit **MIPS** system has 4 GB of addressable space.
  - The **memory** is byte-addressed
  - However, most **load** and **store** operations access **aligned words (4 bytes)**.

MIPS Memory Layout and Addressing

- A standard **MIPS memory** layout divides its program's allocated **main memory** space into five segments:
  - .text: Stores program instructions.
  - .data: Stores static initialized data.
  - bss: Stores static but uninitialized data.
  - **Heap**: Used for dynamic data, growing upwards from the static data segment.
  - **Stack**: Used for function calls and local variables, growing downwards from the top of **memory**.

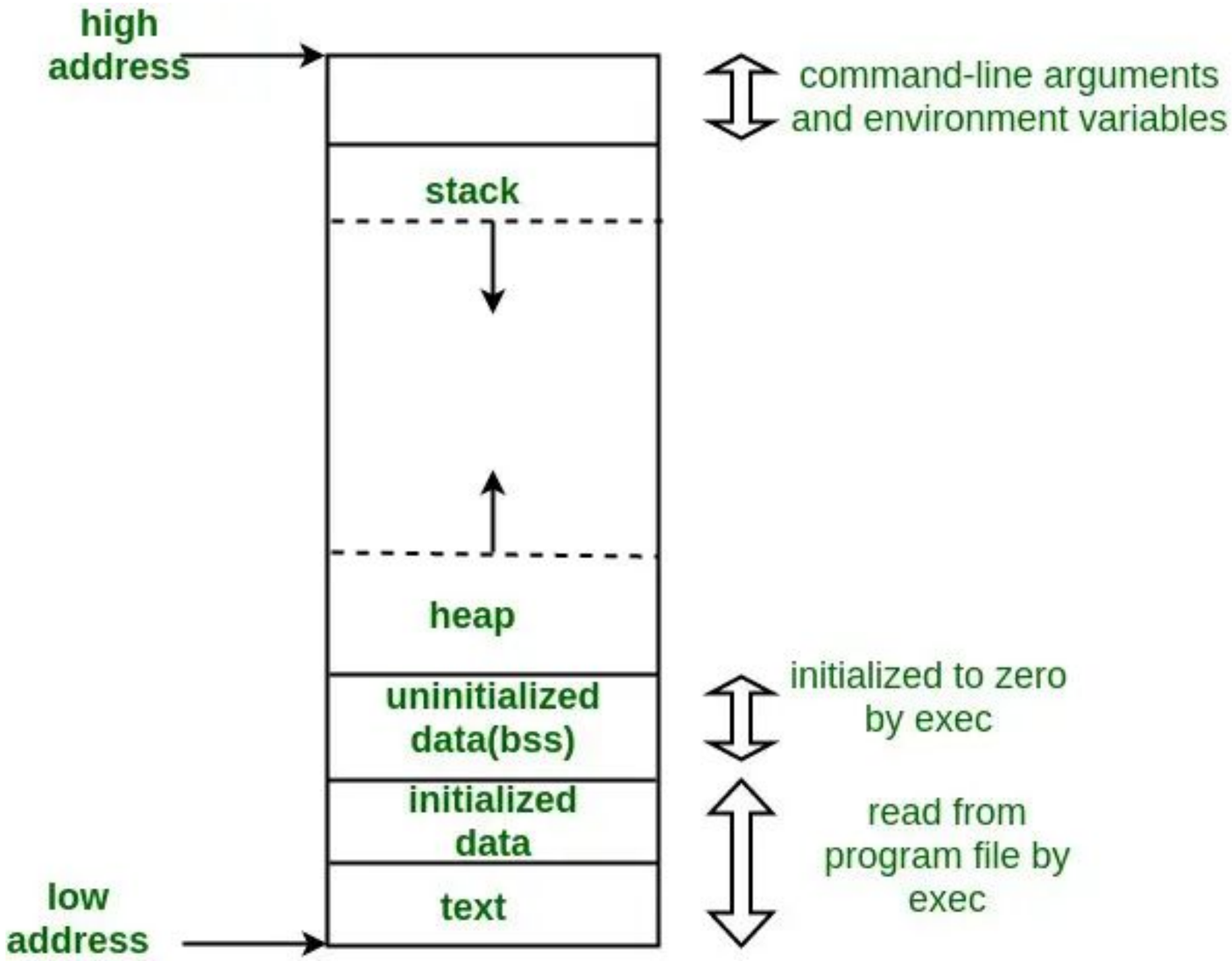
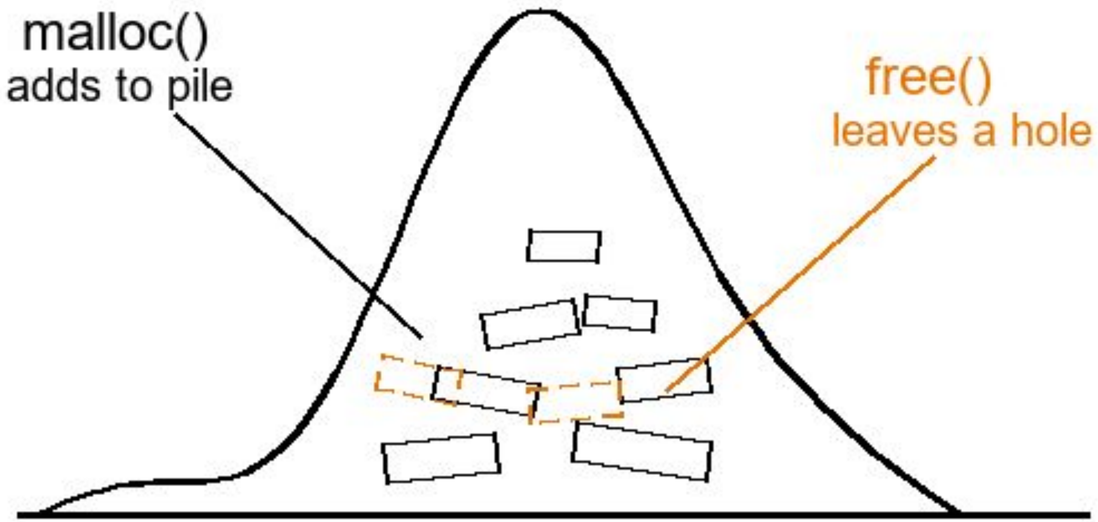


Image Source: https://microcontrollerslab.com/difference-between-stack-and-heap/

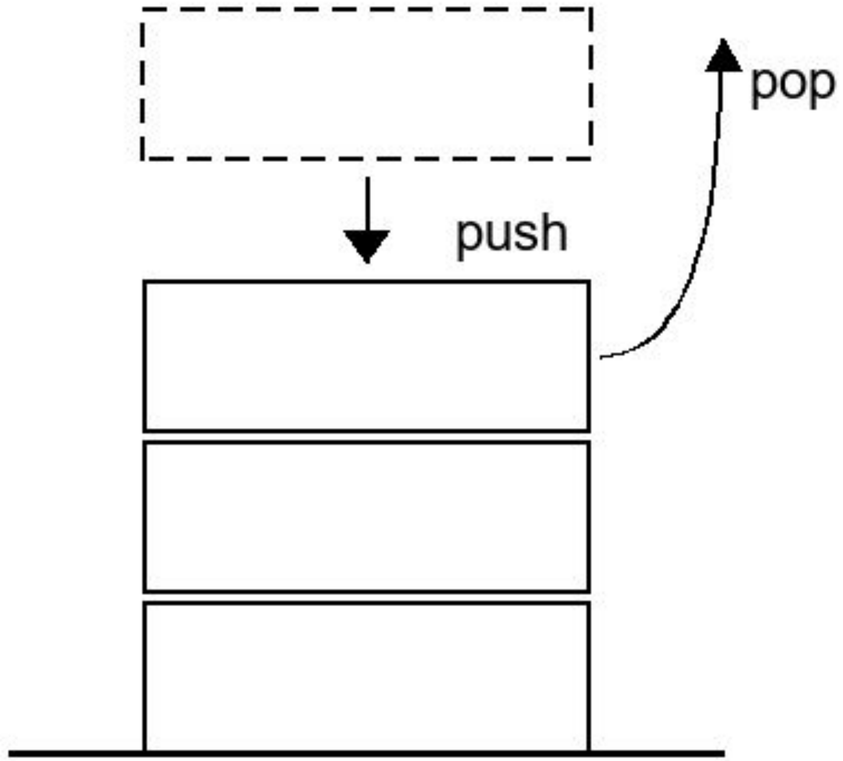
MIPS Memory Layout and Addressing

- **Heap**: Used for dynamic data, growing upwards from the static data segment.



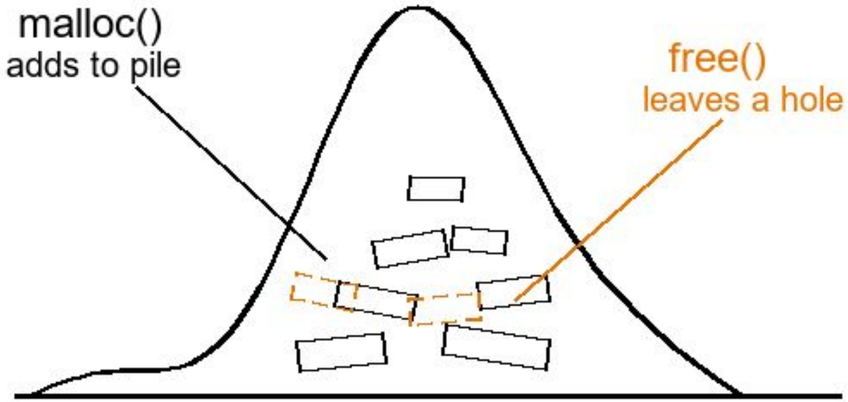
malloc & free are functions in C/ C++.
Assembly uses the same concept.

- **Stack**: Used for function calls and local variables, growing downwards from the top of **memory**.



MIPS Memory Layout and Addressing

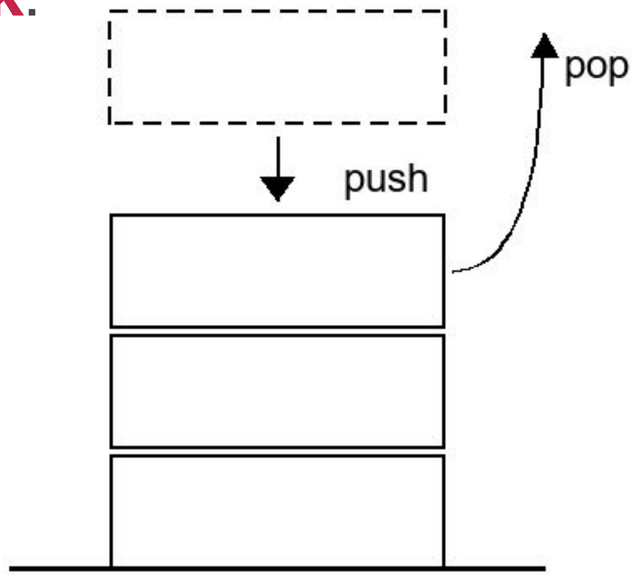
- **Heap**:



malloc	`li $a0, 40`
	`li $v0, 9`
	`syscall`
		Allocates **40 bytes** on the heap
		Address of the block returned in `$v0`

free	*no MIPS equivalent*	Heap memory cannot be freed in MARS In C, `free(ptr)` releases it — leaves a hole in the pile

- **Stack**:



push	`addi $sp, $sp, -4` `sw $ra, 0($sp)`	Stack grows down by 4 bytes Saves `$ra` onto the new space
pop	`lw $ra, 0($sp)` `addi $sp, $sp, 4`	Restores `$ra` from the stack Stack shrinks back up by 4 bytes

MIPS Memory Layout and Addressing

What happens in each code example?





- **Heap**: Used for dynamic data.

```
while (1) {
    malloc(1024 * 1024);
}
```



- **Stack**: Used for function calls and local variables.

```
int fact(int n){
    return n*fact(n-1);
}
```



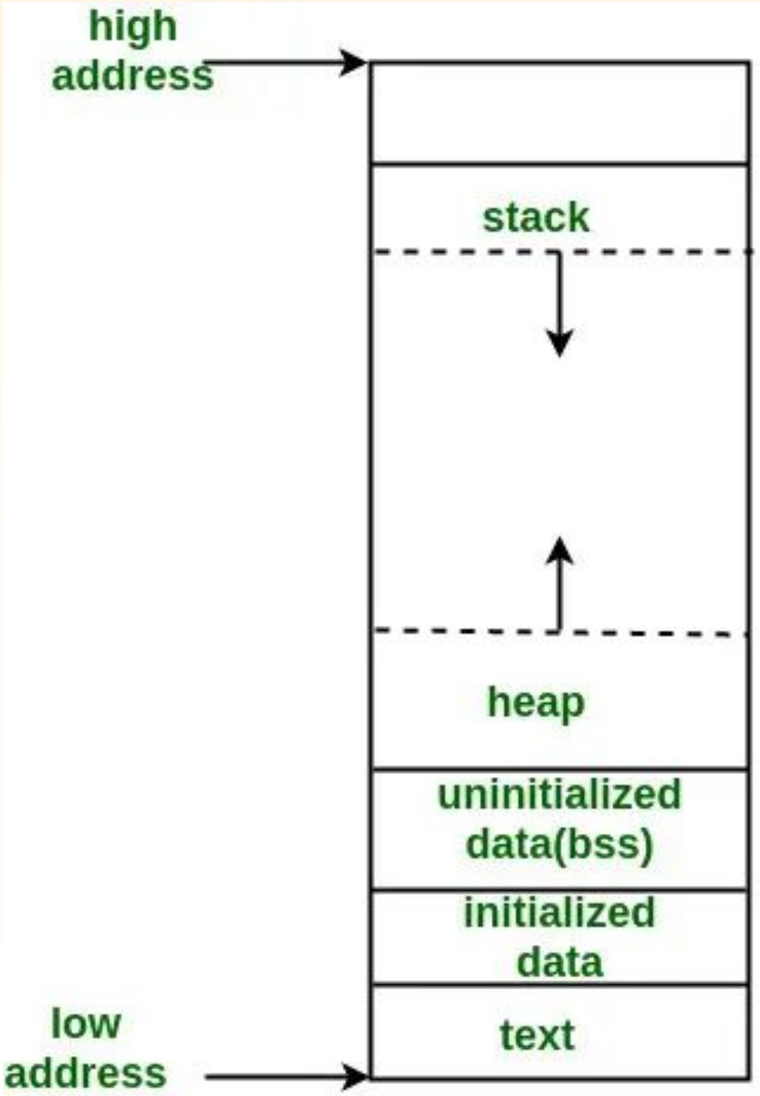


Image Source: https://microcontrollerslab.com/difference-between-stack-and-heap/

MIPS Memory Layout and Addressing

- A 32-bit **MIPS** system has a 4 GB address space.
- The **memory** is byte-addressed, but most **load** and **store** operations access aligned words (4 bytes).

Operate between memory and registers:

- `lw / sw` — load/store a **word (32-bit / 4 bytes)**
- `lh / sh` — load/store a **half-word (16-bit / 2 bytes)**
- `lb / sb` — load/store a **byte (8-bit / 1 byte)**

Operate only on registers:

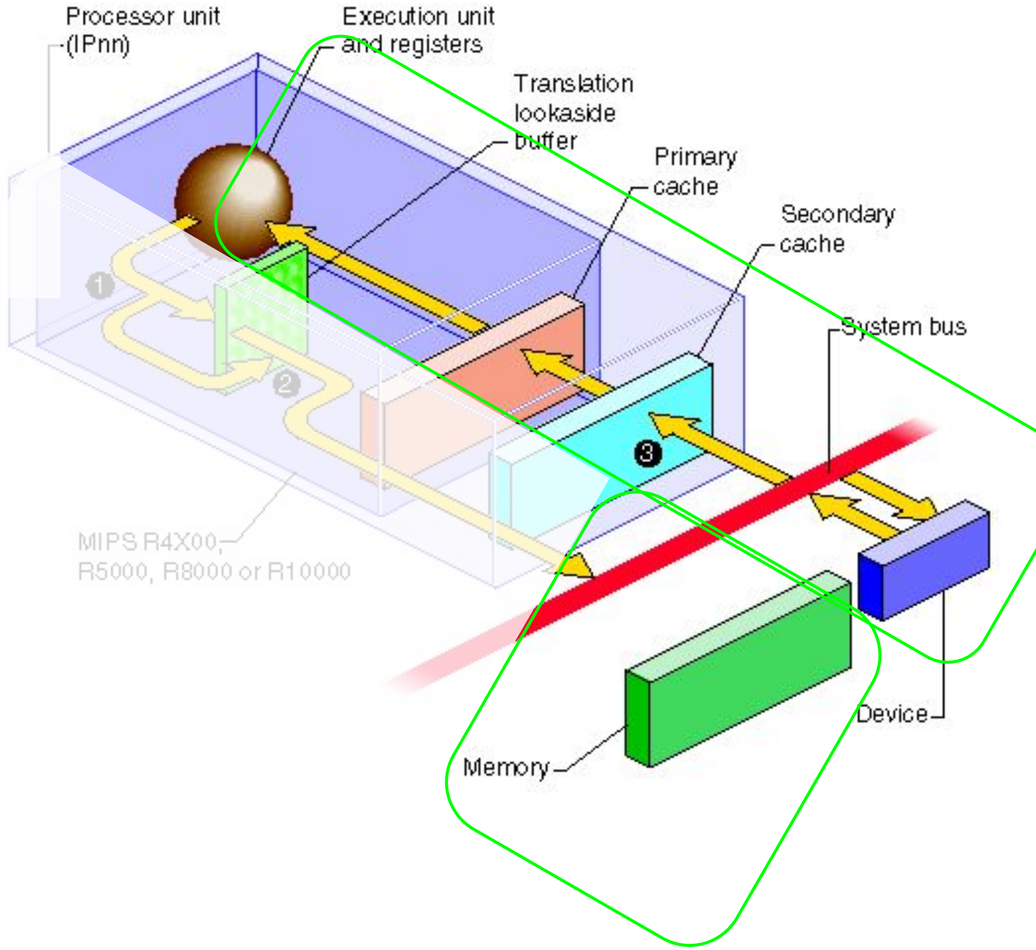
- `li` — load an **immediate constant into a register** (pseudo-instruction, no memory access)
- `move` — copy a value **from one register to another** (no memory access)

Key point:

- `lw/sw, lh/sh, lb/sb` operate between **memory** and **registers** — registers are always one end of every load/store.
- `li` and `move` never touch memory.

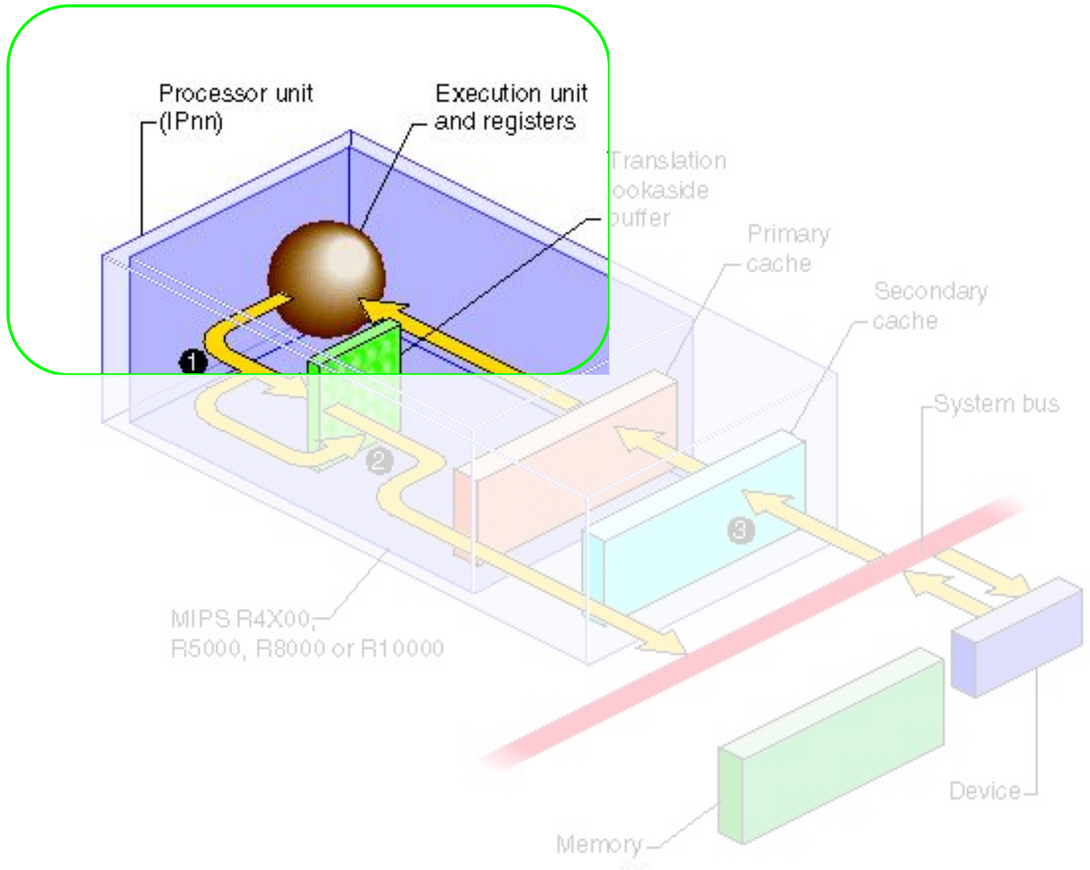
MIPS Memory Layout and Addressing

- **Memory** is accessed using a **"base + displacement" addressing mode**:
  - lw $t0, 8($s0):
    - Loads a word from the **memory** address $s0 + 8 into **register** $t0.
  - sw $t0, 4($s1):
    - Stores the word from **register** $t0 into the **memory** address $s1 + 4.



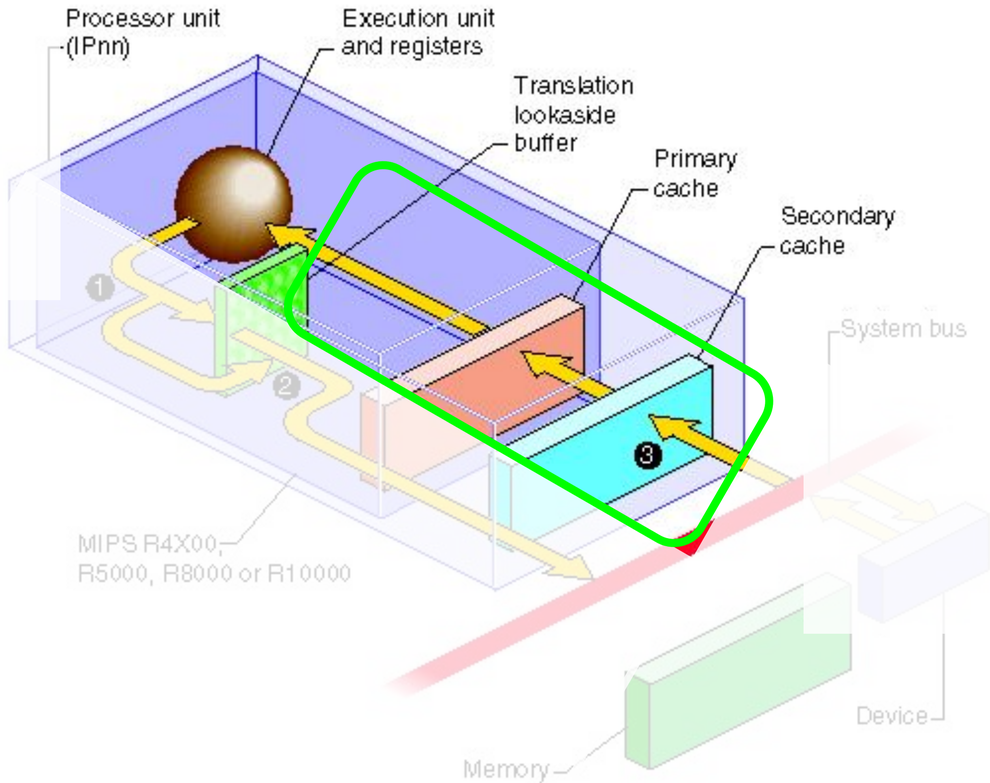
Memory Hierarchy

- Registers
  - **MIPS** has 32 general-purpose registers that are a programmer's primary workspace for data.
  - They provide the fastest possible access to data but are extremely limited in number.



Memory Hierarchy

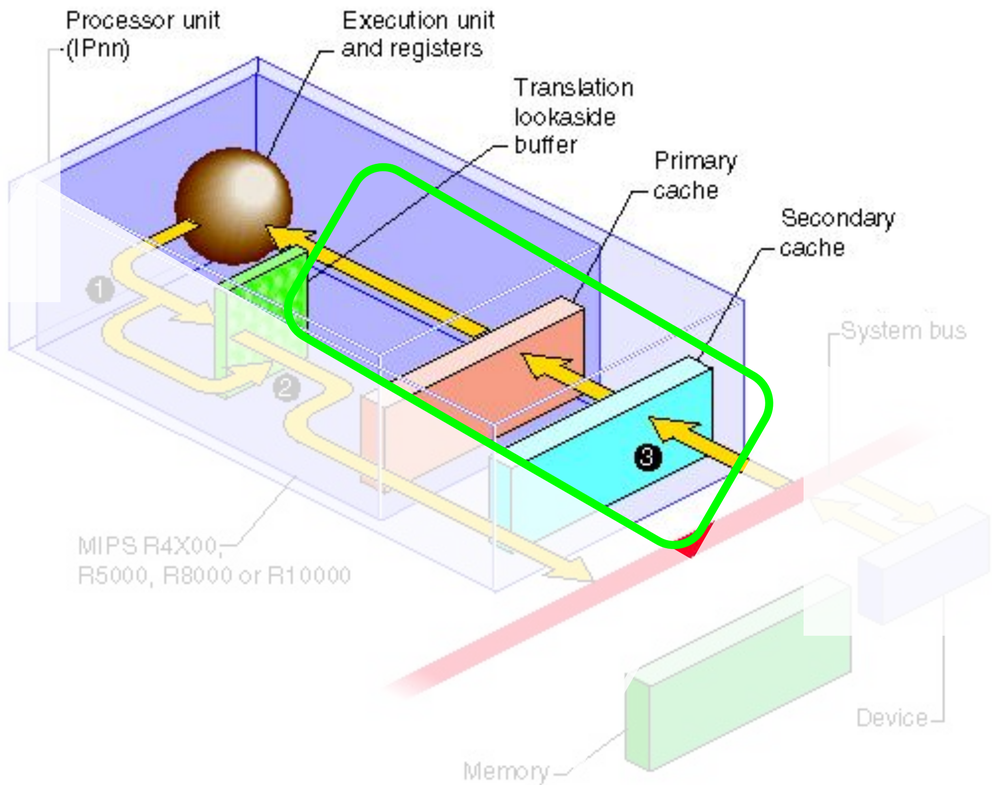
- **Cache memory**
  - Because **main memory is significantly slower than** the **CPU**, **cache memory** is used as a high-speed buffer.
  - The **cache** system is managed by hardware and is generally **transparent to the assembly programmer**.
  - **MIPS** systems **commonly use a split L1 cache**, with separate caches for instructions (i-**cache**) and data (d-**cache**).



Cache: Operation

When the CPU needs a piece of data or an instruction:

1. **Checks L1 cache first.** — every memory access starts at the **fastest cache level** before going anywhere else
2. **Cache Hit:**
   a. If the item is in the cache, it is retrieved very quickly.
3. **Cache Miss:**
   a. **If the item is not found**, the cache controller requests the entire “cache block” or “cache line” containing the data from the next level of memory (e.g., L2 cache or main memory).
   b. The **retrieved block is copied into the cache**, and the requested data is then sent to the CPU.



Note: cache hit rate drives performance. *Why so?*



Cache: write policies

When the **CPU** performs a store operation, the **cache** handles the update according to its write policy:

1. **Write-through:**
    - The data is written to both the cache and main memory simultaneously.
    - This is **simpler to implement** and maintains **consistent memory**, but it is **slower** for writes.

2. **Write-back:** The data is only written to the cache initially.
    - The **cache block** is marked "**dirty**."
    - The *updated data is written back to main memory only when that cache block is evicted* to make room for new data.
    - This offers **better write performance** but **is more complex**.

Impact of Memory on MIPS Assembly Code Design

Understanding the memory hierarchy has several practical implications:

- Registers are important: Maximize the use of registers to hold frequently accessed variables. – They **minimize costly memory access**.

Locality of reference:

- Write code that **exhibits good locality**.
- **Increase cache hits by accessing memory locations** that are
  - close to each other (spatial locality) or
  - accessing the same location multiple times (temporal locality)

Impact on MIPS Assembly

- Understanding the **memory** hierarchy has several practical implications:
  - Optimize loops: Loops are a prime area for exploiting temporal locality.
    - By keeping the working data set small, **a loop can operate mostly within the cache**, drastically improving performance.
  - Instruction vs. Data **cache**: Remember that instructions and data often reside in separate caches.
    - Access patterns for instructions (sequential fetches) are different from data (often random access).
  - Data alignment: **MIPS requires all memory accesses** to be aligned on their natural word boundaries, otherwise an exception is thrown.
    - Programmers must ensure addresses used in lw and sw are multiples of 4.

Examples of MIPS (Applied Memory)

- **MIPS** is a type of **assembly** language that uses simple instructions to perform basic tasks.
- These easy-to-understand examples will introduce fundamental concepts such as arithmetic, **memory access**, and system calls.
- For running and testing **MIPS** code, a simulator like MARS or SPIM is typically used.

Examples of MIPS (Applied Memory)

- Arithmetic operations: C code

```
int num1 = 5;
int num2 = 3;
int result;
result = num1 + num2;
```

EXERCISE

Examples of MIPS (Applied Memory)

```
.data             # Section for data definitions
num1: .word 5     # Declare a word (4 bytes) named num1,
and initialize it with 5
num2: .word 3     # Declare num2 and initialize with 3
result: .word 0   # Declare result and initialize with 0

.text             # Section for instructions
.globl main       # Declare the main label as global

main:
    # Load data from memory into registers
    lw $t0, num1 # Load value of num1 into register $t0
    lw $t1, num2 # Load value of num2 into register $t1
```

```
    # Perform the addition
    add $t2, $t0, $t1
     # Add the values in $t0 and $t1, store in $t2

    # Store result back to memory
    sw $t2, result
     # Store value of $t2 into the memory location 'result'

    # Exit the program
    li $v0, 10  # Load system call code for 'exit' into $v0
    syscall     # Execute system call
```



2 minutes

Why isn't it **displaying** on the screen?
What's the **.word** "data type"?
Can **add** accept **.word** input operands?



Examples of MIPS (Applied Memory)

- Analyzing the **MIPS** Code
  - The .data Section in RAM

```
.data               # Section for data definitions
num1:   .word 5     # Declare a word (4 bytes) named num1, and initialize it with 5
num2:   .word 3     # Declare num2 and initialize with 3
result: .word 0     # Declare result and initialize with 0
...
```

  - When our program is loaded by the operating system, **a section of its memory** (the “data segment”) is allocated in RAM.
  - The assembler directives .word tell the **OS** to reserve space and store the initial values 5, 3, and 0.
  - These three variables live in **RAM** for the entire life of the program.

Examples of MIPS (Applied Memory)

- Analyzing the **MIPS** Code
  - The .text and .globl Sections in RAM

```
...
.text          # Section for instructions
.globl main    # Declare the main label as global
```

```
main:
    lw $t0, num1
    lw $t1, num2
    add $t2, $t0, $t1
    sw $t2, result
    li $v0, 10
    syscall
```

Address	Code	Basic	Source
0x00400000	0x3c011001	lui $1,0x00001001	11: lw $t0, num1 # Load the value of num1 into register $t0
0x00400004	0x8c280000	lw $8,0x00000000($1)	
0x00400008	0x3c011001	lui $1,0x00001001	12: lw $t1, num2 # Load the value of num2 into register $t1
0x0040000c	0x8c290004	lw $9,0x00000004($1)	
0x00400010	0x01095020	add $10,$8,$9	15: add $t2, $t0, $t1 # Add the values in $t0 and $t1, store in $t2
0x00400014	0x3c011001	lui $1,0x00001001	18: sw $t2, result # Store the value of $t2 into the memory location 'result'
0x00400018	0xac2a0008	sw $10,0x00000008($1)	
0x0040001c	0x2402000a	addiu $2,$0,0x0000000a	21: li $v0, 10 # Load the system call code for 'exit' into $v0
0x00400020	0x0000000c	syscall	22: syscall # Execute the system call

  - The program's instructions are also stored in a different section of RAM (the "text segment" or "code segment").
  - When we run the program, the OS tells the **CPU** to start executing instructions from the main: label's address in RAM.

Examples of MIPS (Applied Memory)

- Analyzing the **MIPS** Code
  - Execution - The Dance between **Cache** and Registers
    - This is where the magic happens.
    - The **CPU** doesn't read instructions directly from slow RAM.

Examples of MIPS (Applied Memory)

- RAM is the permanent home for your code and data.
- The **Cache** is a temporary, high-speed copy of the most recently used parts of RAM.
- **Registers** are the **CPU**'s immediate workspace where all actual computation happens.
- **Our program constantly moves data** between **these three levels** to achieve high speed.

Examples of MIPS (Applied Memory)

- Analyzing the **MIPS** Code

Code Part	**Location During Execution**	**Reason**
num1, num2, result values	**RAM**	The program's data is permanently stored here.
Program Instructions	**RAM -> Cache**	Stored in RAM, but copied into the cache line-by-line as the CPU needs to execute them.
lw (**load**) operations	**RAM -> Cache -> Registers**	**Data is fetched from RAM, cached for speed, and placed into a register for the CPU to use.**
add operation	**Registers**	The CPU performs arithmetic only on data already inside its registers.
sw (**store**) operation	**Registers -> Cache -> RAM**	**Data is written from a register to the cache, and later flushed back to RAM.**

Examples of MIPS (Applied Memory)

- Analyzing the **MIPS** Code
  - Execution - Here's the step-by-step process:

1. lw $t0, num1 (Load Word):
    - The CPU needs the value at the **memory address num1**.
    - It first **checks its cache** for a copy of that memory address.
    - On the first access (**a cache miss**), the data is not in the cache. The CPU must **fetch from RAM** for the value 5.
    - The value 5 is brought from RAM into a CPU register ($t0).
      A copy is placed in the cache.

Examples of MIPS (Applied Memory)

2. lw $t1, num2 (Load Word):

   - Same process.
   - The **CPU** checks the **cache** for the address num2.
   - It's likely a miss again, so it goes to RAM, fetches 3, and puts it in **register** $t1 and also into the **cache**.

Examples of MIPS (Applied Memory)

3. add $t2, $t0, $t1 (Add):

Location: **Registers**

- This **operation entirely in** the **CPU**.
- The values are already in the registers $t0 and $t1.
- The **CPU** performs **addition (5 + 3)** and **places result (8)** into **register**, $t2.
- This is the fastest possible operation; it doesn't involve RAM or **cache** at all.

Examples of MIPS (Applied Memory)

4. sw $t2, result (Store Word):

   - **CPU** wants to write value 8 (**register** $t2) to **memory** address result.
   - **CPU** will update the value in the **cache** first (this is very fast).
   - Eventually, the **cache** system will write this updated value back to the main RAM to keep everything in sync.
   - This **write-back** to RAM **might happen later**, not necessarily at this exact moment.

Examples of MIPS (Applied Memory)

```
.data            # Section for data definitions
num1: .word 5    # Declare a word (4 bytes), initialize = 5
num2: .word 3    # Declare num2 and initialize with 3
result: .word 0 # Declare result and initialize with 0


.text            # Section for instructions
.globl main      # Declare the main label as global


main:
    # Load data from memory into registers
    lw $t0, num1     # Load value of num1 into register $t0
    lw $t1, num2     # Load value of num2 into register $t1
```

```
    # Perform the addition
    add $t2, $t0, $t1 # Add the values in $t0 and $t1, store in $t2


    # Store the result back to memory
    sw $t2, result  # Store value of $t2 into memory 'result'


    # Exit the program
    li $v0, 10      # Load the system call code for 'exit' into $v0
    syscall         # Execute the system call
```

- **RAM to registers** — lw $t0, num1 and lw $t1, num2 fetch values from the .data segment in RAM into CPU registers, triggering cache lookups (miss on first access, then cached for reuse)
- **Computation stays in registers** — add $t2, $t0, $t1 operates entirely inside the CPU with no memory access, which is why registers are the fastest level of the hierarchy
- **Registers back to RAM** — sw $t2, result writes the computed value from a register back to memory; the cache is updated first, and the write-back to RAM may happen later depending on the cache write policy

Examples of MIPS (Applied Memory)

- Lw and sw

```
...
main:
    # Load data from memory into registers
    lw $t0, num1     # Load the value of num1 into register $t0
    lw $t1, num2     # Load the value of num2 into register $t1


    # Perform the addition
    add $t2, $t0, $t1
    # Add the values in $t0 and $t1, store in $t2
```

```
    # Store the result back to memory
    sw $t2, result
    # Store the value of $t2 into the memory location 'result'

    # Print the result
    li $v0, 1        # System call code for print integer
    move $a0, $t2    # Move result value to $a0 (argument register for syscall)
    syscall          # Execute the print integer system call


    # Exit the program
    li $v0, 10       # Load the system call code for 'exit' into $v0
    syscall          # Execute the system call
```

- **Load/store** — data must be moved from memory into registers (lw) before the CPU can operate on it, and results must be written back (sw); arithmetic (add) happens only in registers
- **Memory segments in action** — .data holds variables in RAM, .text holds instructions; the program bridges both by loading from one and executing in the other.



Key Points: MIPS Memory & Applied Examples

- **MIPS 32-bit: 4 GB address space, byte-addressed, word-aligned**
- Memory layout: text (code), data, stack, and heap segments
- **Load/Store architecture: lw and sw transfer data to/from memory**
- Base+displacement addressing: lw $t0, 8($s0) loads from $s0+8
- **Cache operation: hit (data found) vs miss (fetch from slower level)**
- Write-through vs write-back policies for cache updates
- **Optimization: maximize register use, exploit cache locality**
- Tight loops on small data sets stay in cache for best performance

PART D

Assembly Programming: MIPS : Integer Operations

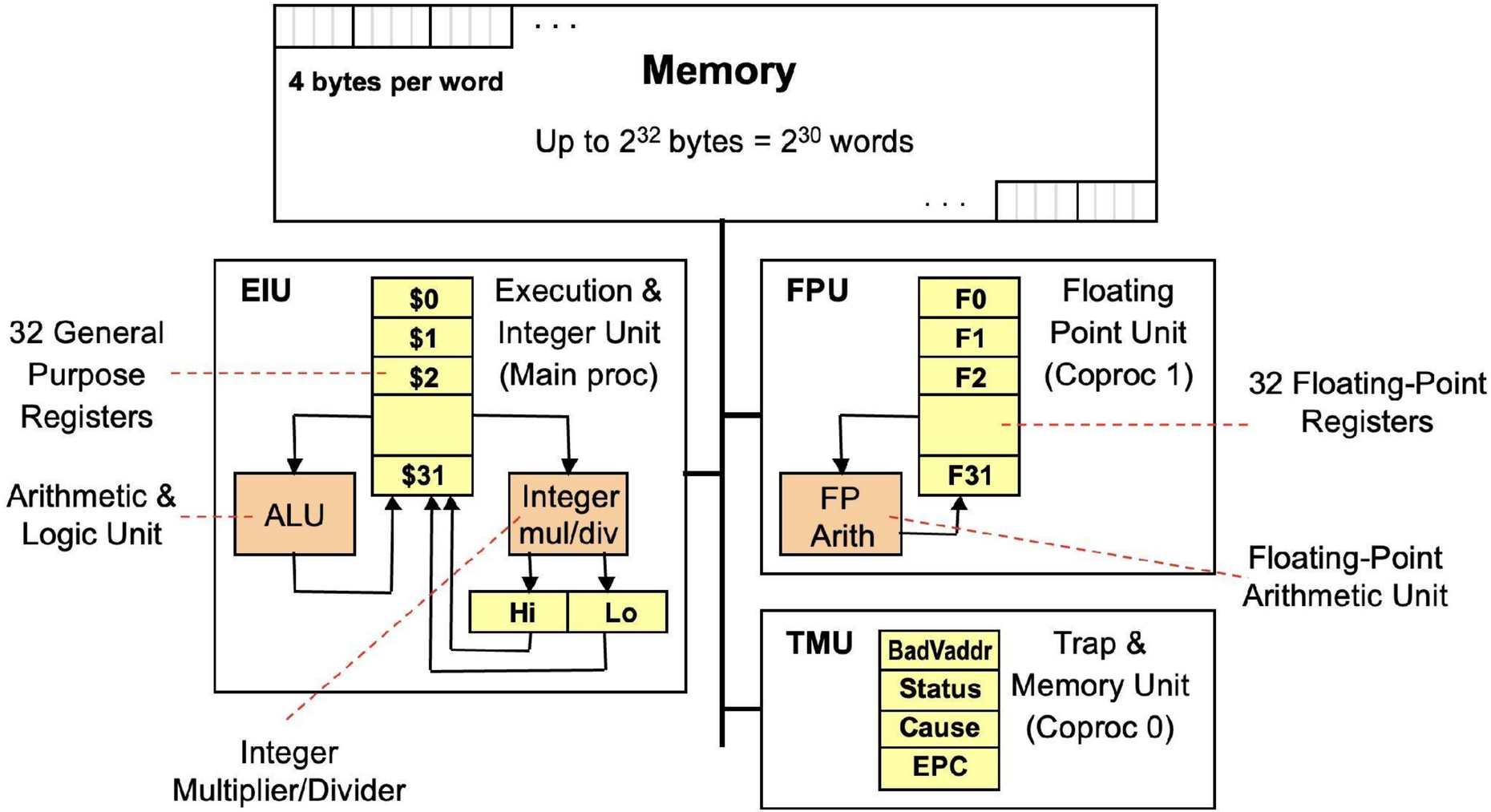
Hands-on MIPS programming with integer operations.

MIPS Instructions Set Architecture *(Reminder)*

- Instructions are the language of the machine
- We will study the **MIPS** instruction set architecture
    - Known as Reduced Instruction Set Computer (RISC)
    - Elegant and relatively simple design
    - Similar to RISC architectures developed in mid-1980's and 90's
    - Very popular, used in many products
        - Silicon Graphics, ATI, Cisco, Sony, etc.
    - Comes next in sales after Intel IA-32 processors
        - Almost 100 million **MIPS** processors sold in 2002 (and increasing).

- Alternative design: Intel IA-32
        - Known as Complex Instruction Set Computer (CISC)

Overview of the MIPS Architecture

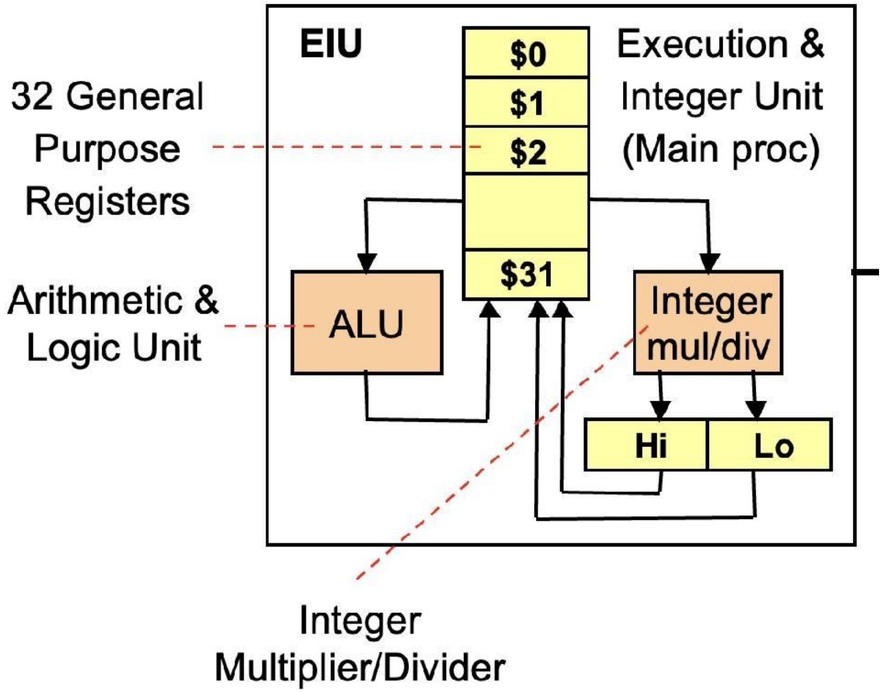


Overview of the MIPS Architecture

Execution & Integer Unit (EIU)

The core block responsible for all integer-based operations:

- **32 General-Purpose Registers:** Fast 32-bit storage ($0 to $31) where register $0 is hardware-wired to always contain the value 0.
- **Arithmetic & Logic Unit (ALU):** Performs primary mathematical (addition, subtraction) and logical evaluations.
- **Integer Multiplier/Divider:** A dedicated hardware unit that processes multiplication and division, outputting results into specialized **Hi** and **Lo** registers.

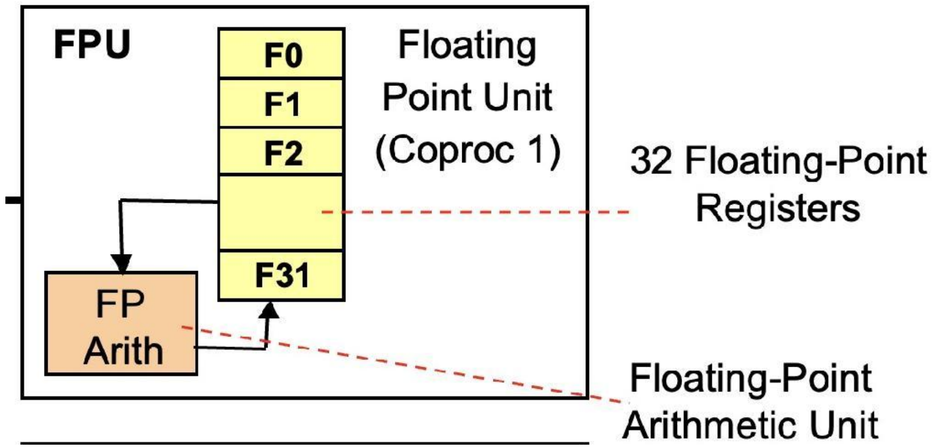


Overview of the MIPS Architecture

Floating-Point Unit (FPU)

A dedicated coprocessor (Coprocessor 1) designed for high-performance floating-point math:

- **32 Floating-Point Registers (F0–F31):** Dedicated 32-bit registers used to store single-precision and double-precision real numbers.
- **Floating-Point Arithmetic Unit:** An independent execution unit that performs arithmetic operations on floating-point operands in parallel with the main CPU.



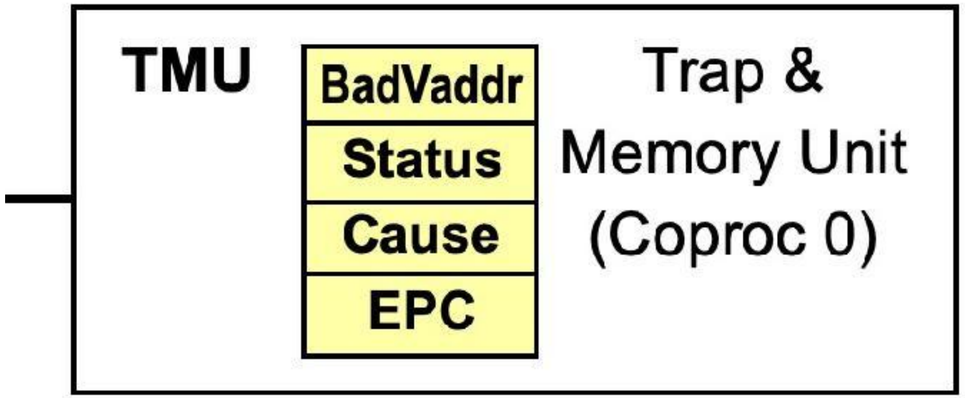
(More on Floating Points in future weeks.)

Overview of the MIPS Architecture

Trap & Memory Unit (TMU)

Coprocessor 0 (Coproc 0) manages system control, exceptions, and virtual memory:

- **System Control & Exceptions:**
  - Serves as Coprocessor 0 to manage **hardware exceptions**, critical for handling **Pipeline Data Hazards**.
    - EPC (Exception Program Counter).
- **Status & Cause Storage:**
  - Stores detailed execution states, including **Carry**, **Borrow**, and **Overflow** **cause** bits.



(More on Carry, Borrow, Overflows in future weeks.)

MIPS Registers:

- 32 general-purpose registers.

Result value registers:
- Named **$v0, $v1**

Argument registers:
- Named **$a0 – $a3**

Temporary registers:
- Named **$t0, $t1 — $t9**

… and a few more.

Variable registers:
- Named **$s0, $s1 — $s7**

#	Name	Notes
0	zero	always contains 0
1	at	reserved for assembler
2	v0	expression evaluation and function results
3	v1	
4	a0	arguments passed to procedures and functions
5	a1	
6	a2	
7	a3	
8	t0	temporary storage – can be overwritten by called routines
…	…	
15	t7	

#	Name	Notes
16	s0	variable storage – must be saved by called routines
…	…	
23	s7	
24	t8	more temporary storage
25	t9	
26	k0	reserved for OS use
27	k1	
28	gp	global area pointer
29	sp	stack pointer
30	fp	frame pointer
31	ra	return address register

Source: https://www.cs.rit.edu/~paw/250/notes/135e/,notes.221/02/MIPS%20registers.pdf

We focus on MIPS in this subject, so you will meet all of these registers.

MIPS General-Purpose Registers

- 32 General Purpose Registers (GPRs)
  - All registers are 32-bit wide in the MIPS 32-bit architecture
  - Software defines names for registers to standardize their use
  - Assembler can refer to registers by name or by number ($ notation)

Name	Register	Usage
$zero	$0	Always 0 (forced by hardware)
$at	$1	Reserved for assembler use
$v0 – $v1	$2 – $3	Result values of a function
$a0 – $a3	$4 – $7	Arguments of a function
$t0 – $t7	$8 – $15	Temporary Values
$s0 – $s7	$16 – $23	Saved registers (preserved across call)
$t8 – $t9	$24 – $25	More temporaries
$k0 – $k1	$26 – $27	Reserved for OS kernel
$gp	$28	Global pointer (points to global data)
$sp	$29	Stack pointer (points to top of stack)
$fp	$30	Frame pointer (points to stack frame)
$ra	$31	Return address (used for function call)

Instruction Categories

- **Integer Arithmetic** (our focus in this presentation)
  - Arithmetic, logic, and shift instructions
- **Data Transfer**
  - Load and store instructions that access memory
  - Data movement and conversions
- **Jump and Branch**
  - Flow-control instructions that alter the sequence

And more to come later …

- **Floating Point Arithmetic**
  - Instructions that operate on floating-point registers
- **Miscellaneous**
  - Instructions that transfer control to/from exception handlers
  - Memory management instructions

MIPS Assembly Programming

- There are three main types of statements that can be used in assembly language, where each statement appears on a separate line:

1. **Assembler directives:** These **provide information to the assembler while translating a program**.
    - Directives are used to **define segments** and **allocate space for variables in memory**.
    - An assembler directive always starts with a dot.
    - Example directives:
        - **.data** Defines the program's data segment, containing the program's variables.
        - **.text** Defines the code segment of the program, containing the instructions.
        - **.globl** Defines a symbol as global that can be referenced from other files.

MIPS Assembly Programming

2. **Executable Instructions:** These generate machine code for the processor to execute at runtime.
   - Instructions tell the processor what to do.

3. **Pseudo-Instructions and Macros:**
   Translated by the assembler into real instructions.
   - These simplify the programmer's task.
   - Example:

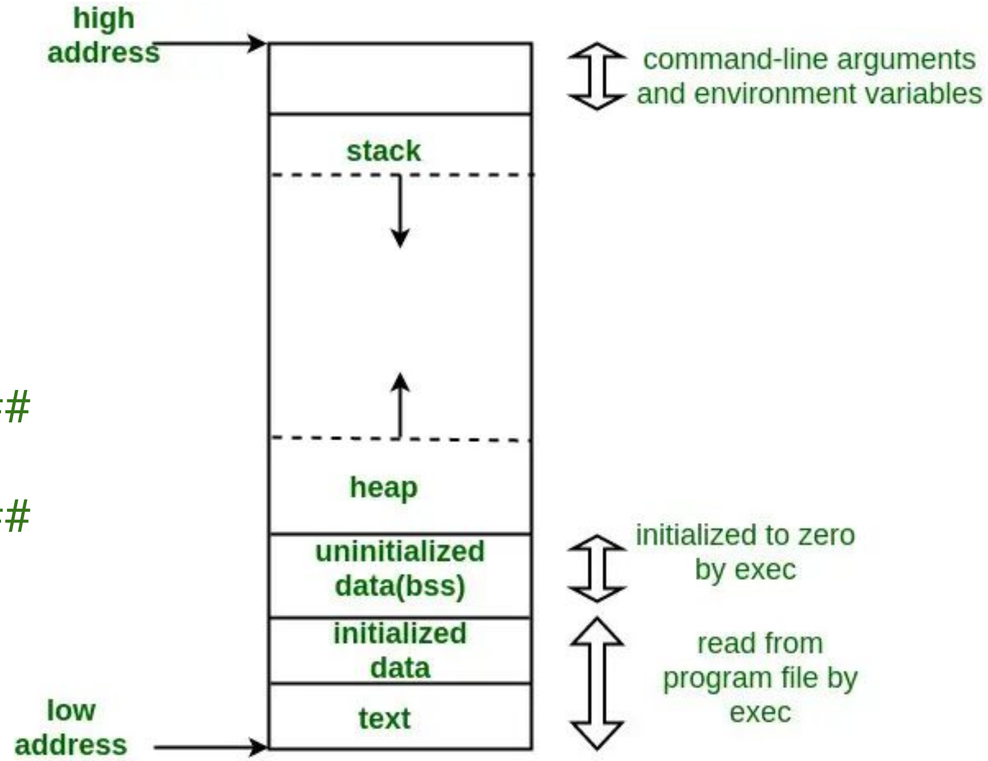
     move —> **addu $rd, $rs, $zero** (**Add the source register to register $zero**)

     (So, add is equivalent to move, because 5+0 = 5).

MIPS Assembly Template

- **MIPS assembly** program template with directives mapping to RAM memory allocations:

```
# Title:
# Author:
# Date:
# Description:
# Input:
# Output:
.data
#################### Data segment ######################
. . .
#################### Code segment ######################
.text
.globl main
main:         # main function entry
. . .
li $v0, 10
syscall       # system call to exit program
```



R-Type Instruction Format: add, addu, …



- Op: operation code (opcode)
  - Specifies the operation of the instruction
  - Also specifies the format of the instruction
- funct: function code – extends the opcode
  - Up to 26 = 64 functions can be defined for the same opcode
  - MIPS uses opcode 0 to define many R-type instructions
- Three Register Operands (common to many instructions)
  - Rs, Rt: first and second source operands
  - Rd: destination operand
  - sa: the **shift amount** used by **shift instructions**

Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Type Integer Add and Subtract

Instruction	Meaning	Op	Rs	Rt	Rd	sa	func
add $t1, $t2, $t3	$t1 = $t2 + $t3	0	$t2	$t3	$t1	0	20
addu $t1, $t2, $t3	$t1 = $t2 + $t3	0	$t2	$t3	$t1	0	21
sub $t1, $t2, $t3	$t1 = $t2 – $t3	0	$t2	$t3	$t1	0	22
subu $t1, $t2, $t3	$t1 = $t2 – $t3	0	$t2	$t3	$t1	0	23

- **add, sub:**
  - arithmetic overflow causes an exception
  - In case of overflow, the result is not written to the destination register.
- **addu, subu:**
  - arithmetic overflow is ignored
  - compute the same result as add, sub

- **Many programming languages ignore** overflow
  - The **+ operator is translated into addu**
  - The **– operator is translated into subu**



But, why?



Key Points: MIPS Architecture & R-Type Instructions

- **MIPS: 32 general-purpose registers, each 32-bit wide**
  - Conventions: $t0-$t9 (temp), $s0-$s7 (saved), $v0 (return value)
- **Assembly template: .data for variables, .text for instructions**
  - Three statement types: directives, instructions, macros
- **R-Type format: opcode | rs | rt | rd | shamt | funct**
  - add/sub cause overflow exceptions; addu/subu do not

PART E

Integer Unsigned
ADD/SUB Operations

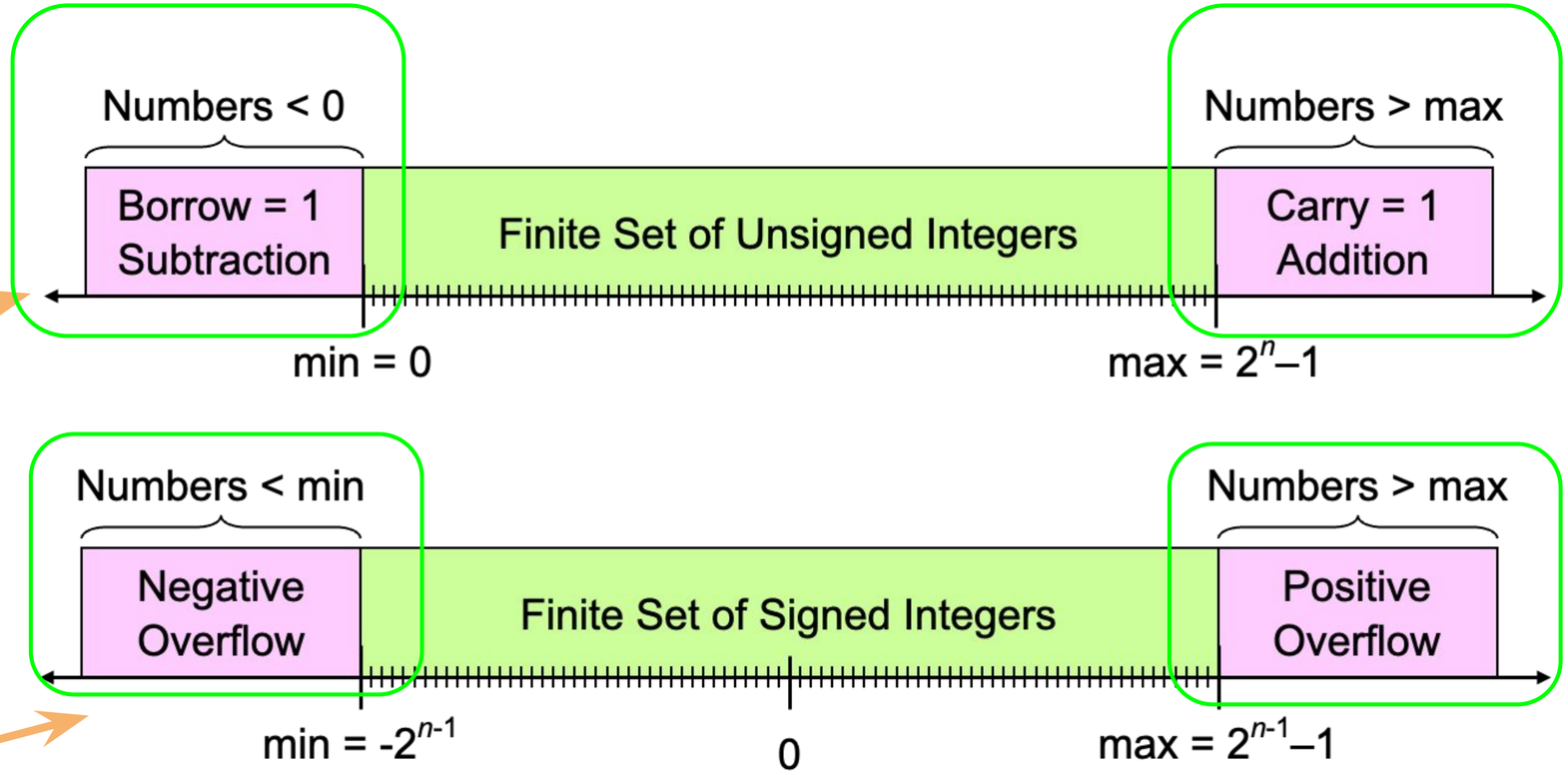
Unsigned integer ranges, addition and subtraction.

Bits have NO meaning:

- The same n bits stored in a **register** can represent an unsigned or a signed integer.

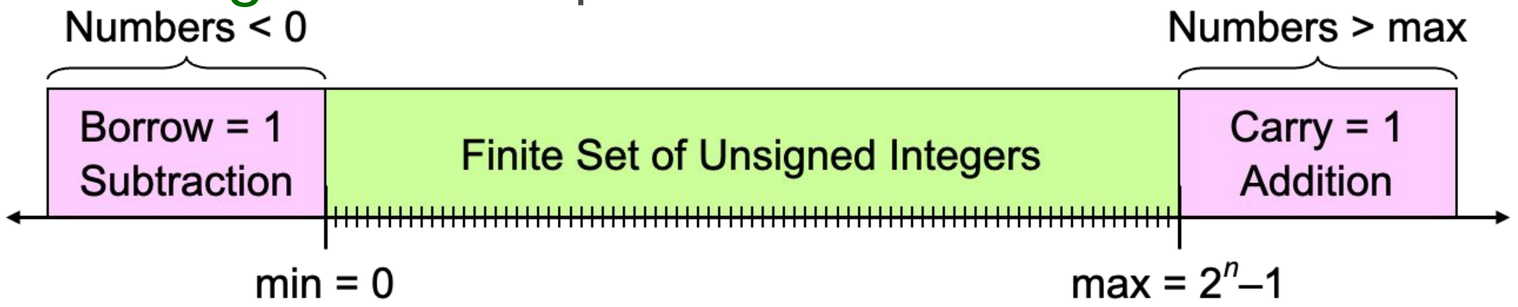
- Unsigned Integers: n-bit representation.
  Borrow Bit
  Carry Bit

- Signed Integers: n-bit 2's complement representation
  Positive Overflow
  Negative Overflow



Range, Carry, Borrow, and Overflow

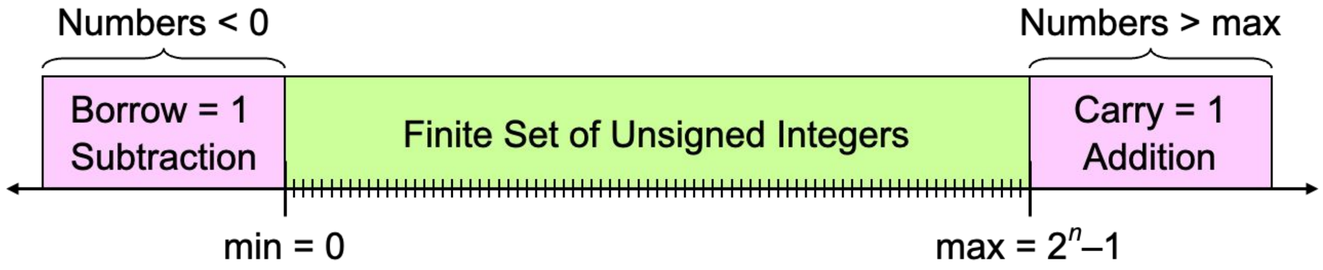
- n-bit Unsigned Integers: n-bit representation



- The "Finite Set" is the range of numbers.
- For n bits, it can represent numbers from 0 to 2^n - 1.
  - For example (4-bit): The range is 0 (0000) to 15 (1111).

Range, Carry, Borrow, and Overflow

- n-bit Unsigned Integers: n-bit representation



- "Carry = 1 for Addition" happens when adding 1 to the largest possible number.
  - MAX_VALUE + 1 "wraps around" to 0.
- In binary addition, this operation generates a Carry-out bit of 1 from the most significant bit (MSB).
- The result in the n-bit **register** is 0.
  - For example (4-bit): 15 (1111) + 1 = 16 (10000).
    - The 4-bit **register** can only hold 0000, so the result is 0.
    - The 1 is the carry flag.

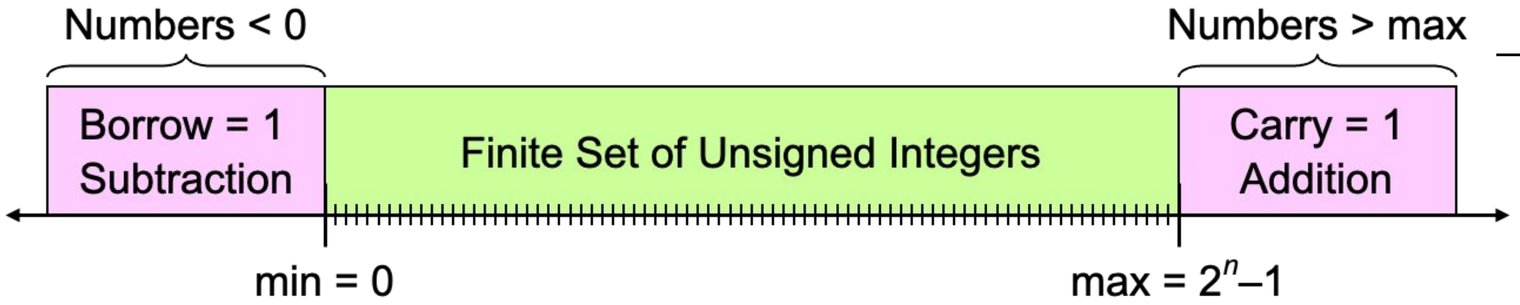
```
  0b1111
+ 0b0001
--------
 0b10000
carry 1  0b0000
```

Range, Carry, Borrow, and Overflow

```
  0b0000
- 0b0001
---------
  0b10000
- 0b0001
---------
  0b1111
```

borrow 1

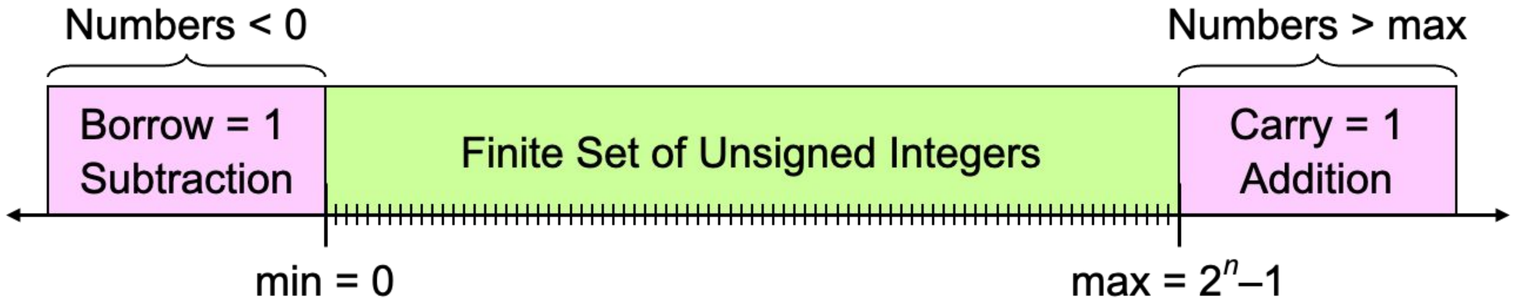
- n-bit Unsigned Integers: n-bit representation



- "Borrow = 1 for Subtraction" happens when subtracting 1 from the smallest possible number (0).
  - 0 - 1 "wraps around" to MAX_VALUE.
- In binary subtraction, this operation requires a Borrow bit of 1.
- The result is the maximum value.
  - For example (4-bit): 0 (0000) - 1.
    - Think of it as borrowing, making it 16 (10000) - 1 = 15 (1111) in the 4-bit **register**.
    - The result is 15.

Range, Carry, Borrow, and Overflow

- n-bit Unsigned Integers: n-bit representation



  - The number line is a circle.
  - Counting up past the maximum wraps to the minimum.
  - Counting down past the minimum wraps to the maximum.
  - This is often called an **unsigned overflow** or wrap-around.



Key Points: Unsigned Integer Range

- **n-bit unsigned integers: range 0 to 2^n - 1**
  - 8-bit: 0-255; 16-bit: 0-65,535; 32-bit: 0 to ~4.29 billion
- **Carry flag signals unsigned result exceeds representable range**
  - Carry is the bit overflowing past the MSB position

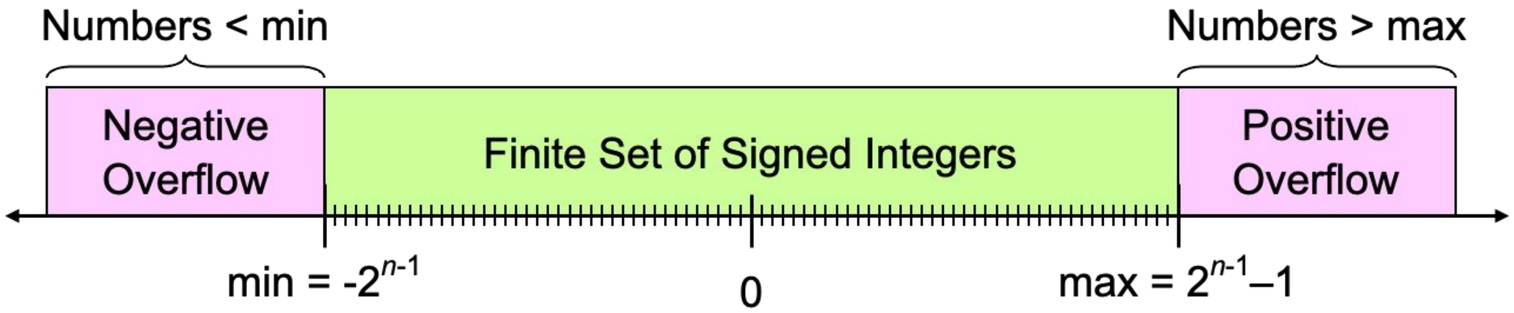
PART F

Integer Signed
ADD/SUB Operations

Signed integers in two's complement: add and subtract safely.

Range, ~~Carry, Borrow,~~ and Overflow

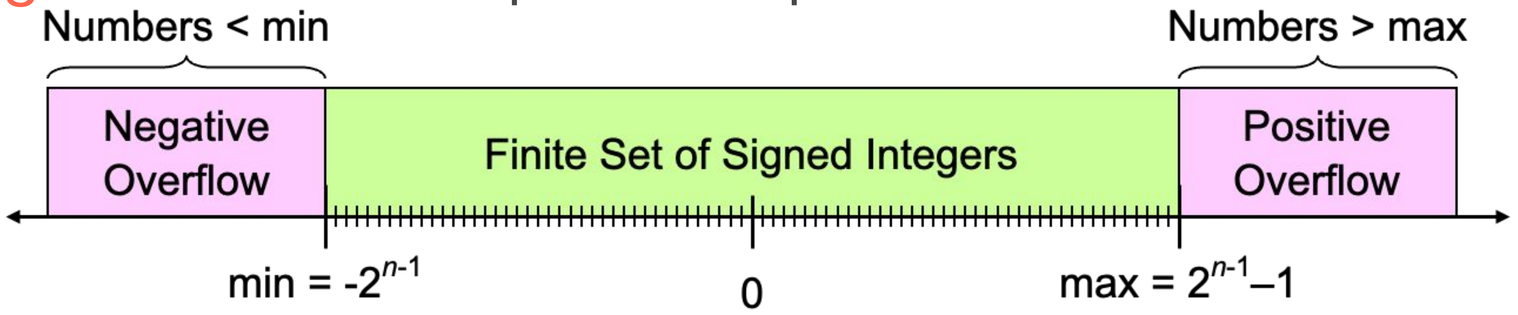
- Signed Integers: n-bit 2's complement representation



- The "Finite Set" is the range of numbers.
- For n bits, it can represent numbers from -2^{n-1} to $2^{n-1} - 1$.
  - For example (4-bit): The range is -8 (1000) to 7 (0111).

Range, Carry, Borrow, and Overflow

- Signed Integers: n-bit 2's complement representation



- The number line is also a circle, but the "join" or wrap-around point is between the most positive and most negative numbers, not at zero.
- **Overflow** is defined based on whether the sign bit changed incorrectly.
  - **Positive Overflow**: A positive number became negative.
  - **Negative Overflow**: A negative number became positive.

Two's Complement

In binary representation, particularly in the two's complement system, which is the standard for representing signed integers in computers.

Two's Complement Basics:

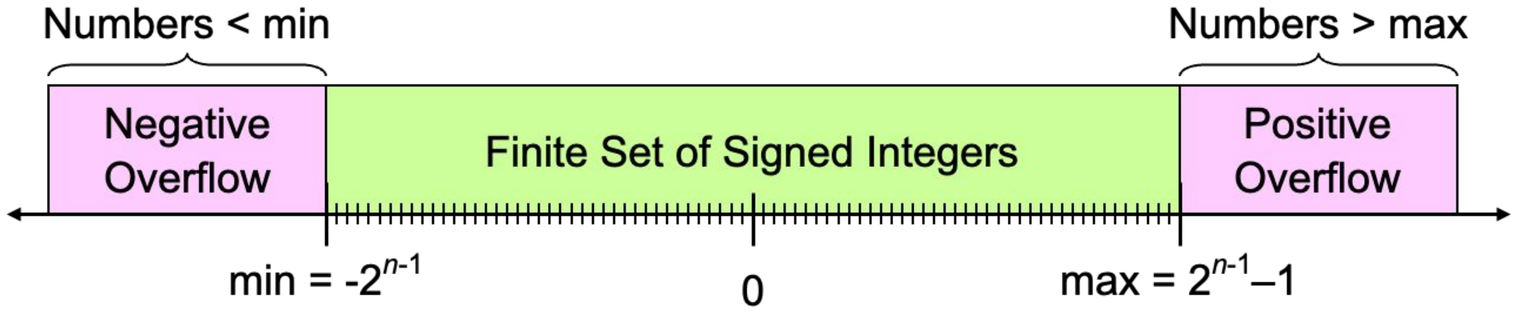
- In t**wo's complement**, the **most significant bit (MSB) is the sign bit:**
  - 0 = positive number
  - 1 = negative number
- To represent a negative number, we:
  - Start with the positive binary representation.
  - Invert all the bits (this is the one's complement).
  - Add 1 to the result (to get the two's complement).

For example: -1

```
0b0001
0b1110 +1
0b1111
```

Two's Complement: Positive Overflow

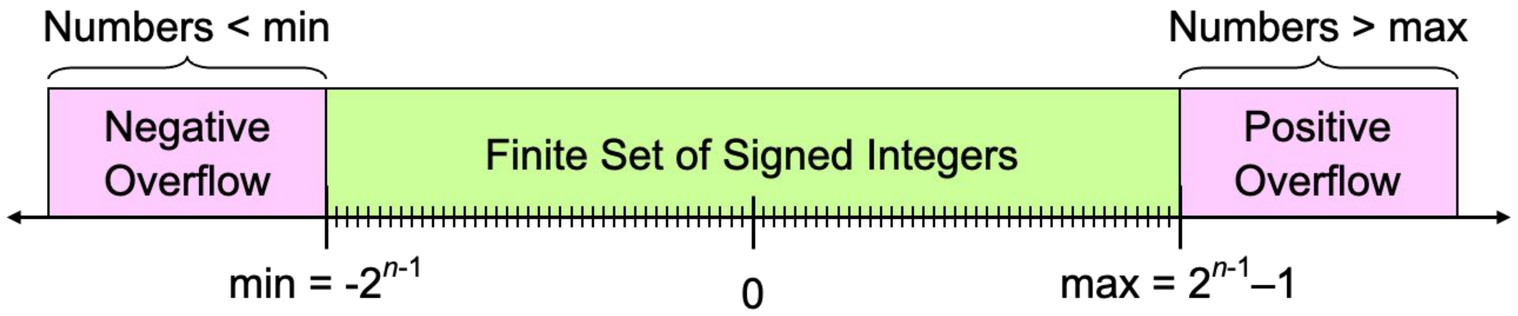
- Signed Integers: n-bit 2's complement representation



- "**Positive Overflow**" happens when adding a positive number to the largest positive number, pushing it past the maximum (wraps to the minimum).
- MAX_POSITIVE + 1 becomes the MOST_NEGATIVE number.
- Adding to 0111...1 (max positive) flips the bits to 1000...0 (the most negative number), and the sign bit changes incorrectly.
  - For example (4-bit): 7 (0111) + 1 = 8.
    - But 8 is 1000 in binary, which is -8 in 2's complement.
    - The result is -8, not 8.

	+0b0111	7
	+0b0001 +	1
positive **overflow**	+0b1000	8
convert to -8	-0b1000	-8

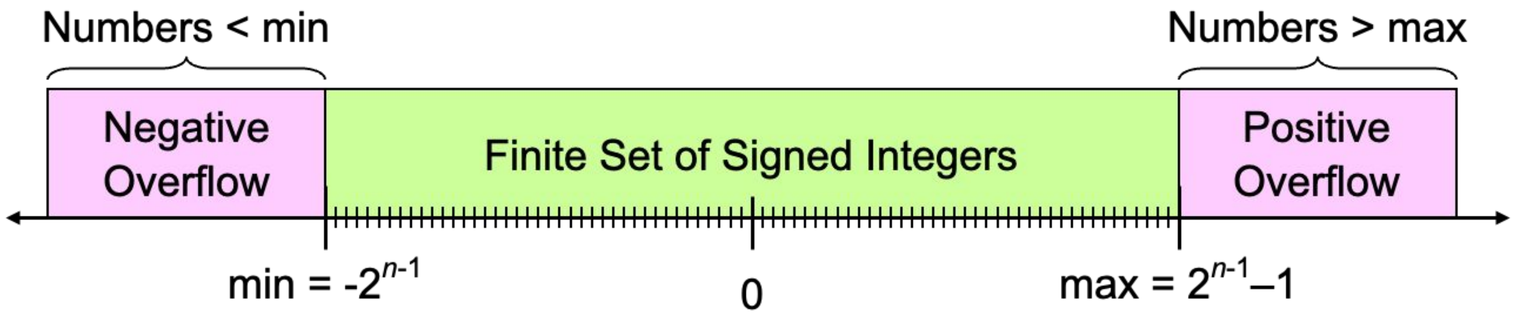
Two's Complement: Positive Overflow




```
                +0b0111   7
                +0b0001 + 1
               ---------
positive overflow +0b1000   8
               ---------
   convert to -8  -0b1000  -8
```

Two's Complement: Negative Overflow

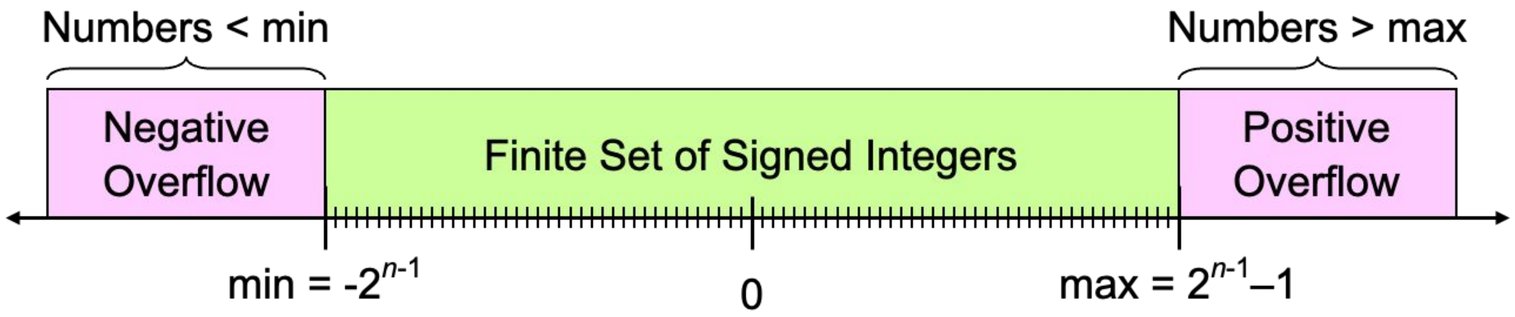
- Signed Integers: n-bit 2's complement representation



- "**Negative Overflow**" happens when subtracting from the most negative number, pushing it past the minimum (wraps to the maximum).
  - MOST_NEGATIVE - 1 becomes the MAX_POSITIVE number.
  - Subtracting from 1000...0 (most negative) flips the bits to 0111...1 (the max positive number), and the sign bit changes incorrectly.
    - For example (4-bit):* -8 (1000) - 1 = -9.
      - But -9 would be 10111 in more bits.
      - In 4 bits, we get 0111 (7) because the calculation wraps around.

Two's Complement: Negative Overflow

- Signed Integers: n-bit 2's complement representation



For example: -1

```
0b0001
0b1110 +1
0b1111
```

negative **overflow**

convert to 7

```
-0b1000  -8
-0b1111  -1
---------
-0b10111 -9
 0b0111   7
```

Carry & Overflow

- Carry is useful when adding (subtracting) unsigned integers.
  - Carry indicates that the unsigned sum is out of range.
- **Overflow** is useful when adding (subtracting) signed integers
  - **Overflow** indicates that the signed sum is out of range.
- Range for 32-bit unsigned integers = 0 to $(2^{32} - 1)$
- Range for 32-bit signed integers = -2^{31} to $(2^{31} - 1)$



Key Points: Signed Integers & 2s Complement

- **2's complement range: -2^(n-1) to 2^(n-1) - 1**
  - MSB is the sign bit: 0 = positive, 1 = negative
- **Wrap-around: MAX_POSITIVE + 1 becomes MOST_NEGATIVE**
  - Number line forms a circle at the positive-negative boundary
- **Overflow: sign bit changes incorrectly after operation**
  - Carry is for unsigned; Overflow is for signed arithmetic

PART G

Integer Carry/Overflow Examples

Worked examples of carry and overflow in binary arithmetic.

Carry & Overflow

- For example 1:
  - Carry = 1, **Overflow** = 0 (NO overflow)
  - Unsigned sum is out of range, but the signed sum is correct

```
      11111 1               1 11      1
A:    1000 0100 0000 0000 1110 0001 0100 0001
+
B:    1111 1111 0000 0000 1111 0101 0010 0000
      ---------------------------------------
Sum:  1000 0011 0000 0001 1101 0110 0110 0001
```

Unsigned sum is out of range, but the signed sum is correct. Both numbers are huge (since the MSB is 1).
A = a large positive number (above 2^{31}
B = a large positive number (above 2^{31}
Their sum will exceed 2^{32} - 1, the max unsigned 32-bit value, so we expect **Carry = 1**.

As Signed Integers (Two's Complement):
MSB=1 means both are negative.
A = a negative number
B = a negative number
Adding two negatives should yield a negative result. Since the result is within the signed range, we expect **Overflow = 0**.

Carry & Overflow

- For example 2:
  - Carry = 0, **Overflow** = 1
  - Unsigned sum is correct, but the signed sum is out of range.

```
          01111 1                  11    1
A:      0010 0100 0000 0100 1011 0001 0100 0100
+
B:      0111 1111 0111 0000 0011 0101 0000 0010
        ---------------------------------------
Sum:    1010 0011 0111 0100 1110 0110 0100 0110
```

Let's check the MSB (bit 31) of the operands and the result:

- Number A: MSB = 0 (positive)
- Number B: MSB = 0 (positive)
- Result: MSB = 1 (negative)

Since both operands are positive but the result is negative, this indicates an **overflow**. Therefore, Overflow = 1.



Key Points: Carry & Overflow Examples

- **Carry=1, Overflow=0: unsigned out of range, signed correct**
- **Carry=0, Overflow=1: unsigned correct, signed out of range**
- **Carry and overflow are independent conditions**

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025